
EFFICIENT LARGE LANGUAGE MODELS VIA RECURRENT TRANSFORMER BLOCKS

Tobias Kerner

Technische Hochschule Ingolstadt

<https://orcid.org/0009-0008-5485-1018>

ABSTRACT

This thesis introduces modifications to a novel language model architecture. The architecture contains multiple recurrent blocks, each featuring multiple sandwich transformer blocks. We train a language model of this architecture with 308M parameters and evaluate it against similarly and larger sized models on popular benchmarks.

1 INTRODUCTION

Large Language Models, or AI in general, have a direct impact on society. From automation of jobs to entertainment, creative writing or language learning, the usage and applications of AI systems are growing rapidly with widespread industry adoption (Figure 1).

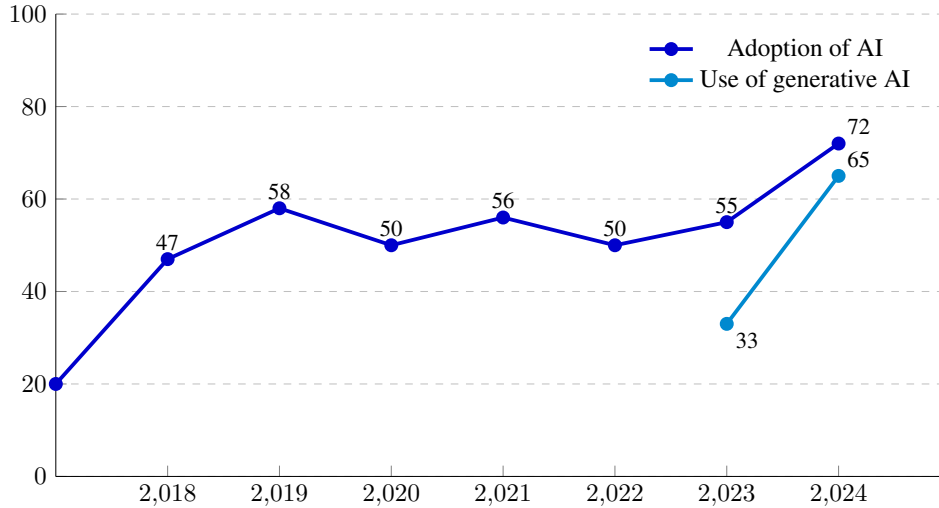


Figure 1: Organizations that have adopted AI in at least 1 business function. Data shows percentage of respondents from McKinsey Global Survey on AI (2017–2024). Definition of AI adoption has evolved: 2017 used “AI in core business or at scale”; 2018–2019 used “embedding at least 1 AI capability”; since 2020 defined as “adopted AI in at least 1 function.” Source: McKinsey & Company. (Singla et al., 2024)

Many people use convenient APIs from model providers such as Anthropic’s Claude, Google’s Gemini and OpenAI’s ChatGPT, besides which it is also possible to run and host smaller AI models and LLMs on consumer hardware for local inference. Popular model sizes that run on consumer hardware range from 7B to 32B parameters, with Alibaba’s Qwen (Yang et al., 2025) and Meta’s Llama models (Grattafiori et al., 2024) dominating in popularity in open weight and local inference related communities at the time of writing. Larger and more intelligent models, even if open-weight versions are available, can usually only be hosted on expensive datacenter-GPUs and usable via

API. These models can reach sizes between 200B and up to 1T parameters, possibly even higher for proprietary ones. Examples of such very large open weight models are Kimi-K2 (Team et al., 2025), Deepseek-v3 (DeepSeek-AI et al., 2025), and Qwen3-235B-A22B (Yang et al., 2025).

A limiting factor to the benefits of AI, however, are model sizes and resource requirements, such as the cost of compute, for both training and inference. Pretraining state-of-the-art language models requires massive Datacenters, GPU-hours, and electricity consumption. The training of LLaMA-65B, for example, was done over 1,022,362 A100-80GB GPU-hours, consuming 449 MWh of energy and causing emission of approximately 173 tons of CO₂ (Touvron et al., 2023). Even after the expensive training of the model, inference is still computationally expensive, especially for reasoning models which scale output quality through test-time compute. One of the first publically available ‘thinking’ models being OpenAI-o1 (OpenAI, 2024). The efficiency of training or running language models dictates who can train their own model, or run it locally. Lower cost and energy requirements can enable smaller companies or groups of people to train models privately on their own data, with lower memory requirements allowing better performance and lowering the entry barrier. This allows for a wider range of use cases that require local inference with limited resources.

Traditionally, transformers scale by adding more layers or increasing internal dimensions, such as the embedding dimensions or the intermediate size in multi-layer-perceptron blocks. Higher model capacity and output quality come at the cost of reduced computational efficiency and higher memory footprint, reducing accessibility and limiting usage. GPT-3 for example was trained with 175B parameters (Brown et al., 2020), while recent models exceed up to 1T parameters. Training such large models requires thousands of GPUs over months, consuming high amounts of energy and producing significant carbon emissions. This makes training very expensive, costing large amounts of money, and limits it to only well funded organizations. Furthermore, standard transformer models lack recurrent processing, meaning that each parameter inside the model is applied at most once during a forward pass when processing input data.

In their work, which this research builds upon, Geiping et al. (2025) introduce a novel architecture using a recurrent block, containing multiple transformer blocks, which iterates multiple times over a latent tensor. With this, they aim to improve parameter efficiency by using each parameter in the recurrent block multiple times during the forward pass to process data. They show that with this architecture, they can achieve similar or better performance on select benchmarks compared to other models, such as OLMo-7B (Groeneveld et al., 2024), with higher parameter count. While this architecture benefits from lower memory requirements, the single big monolithic recurrent block brings some limitations. The long chain of recurrent transformer blocks, even with residual connections, might result in a suboptimal gradient flow and therefore lower training stability, as well as lower flexibility in terms of controlling how many times a set of parameters operates on a latent tensor and contributes to the output.

In this work, we propose an architectural change to Geiping et al. (2025) implementation, by splitting the single large recurrent block into multiple smaller recurrent blocks. Instead of using one recurrent block containing 12 transformer blocks, for example, we test how distributing the transformer blocks over multiple recurrent blocks, e.g. 3 recurrent blocks of 4 transformer blocks each, affects training and evaluation performance. The key change in our implementation is the modular recurrent architecture, allowing dynamic specification of number of recurrent blocks, with a separate number of transformer blocks in each. Furthermore, during inference it is possible to specify the number of recurrent steps for each recurrent block individually, making it possible to test the effect of specific recurrent blocks on certain tasks and efficiently prioritizing compute for sections of the model. We analyze how the splitting into multiple recurrent blocks affects training stability, what performance can be achieved with this architecture in benchmarks, and in what way different iteration counts of each recurrent block in the model affect performance.

Summary

- Novel multi-block recurrent transformer architecture
- Experimental validation on datasets and benchmarks
- Analysis of training dynamics and convergence
- Comparison to other models

The remainder of this work is structured as follows: Section 2 reviews related work on transformer architectures, scaling laws, normalization techniques, and recurrent approaches, with specific focus on the work of Geiping et al. (2025). Section 3 describes the used architecture with multiple recurrent transformer blocks, going into detail about the modular design and architectural components. Section 4 describes our experimental setup, training method, training history analysis, and benchmark evaluations; including example outputs of the trained model for given prompts. Section 5 concludes with the key findings of our experiments and potential directions for future work.

2 RELATED WORK

In this section, we will first go through the foundations of the transformer architecture and take a look at propositions for scaling laws of transformer models. After this, we briefly analyze parameter sharing approaches and normalization techniques, followed by an explanation of the truncated back-propagation through time (TBPTT) technique and end with an overview of the proposed architecture by Geiping et al. (2025), which this research builds upon.

2.1 NOTATION REFERENCE

The following tables provide a reference to mathematical notation used in this section.

Scalars

Table 1: Scalar notation

Symbol	Description
T_{seq}	Sequence length (number of tokens)
T_{total}	Total recurrent iterations (in TBPTT context)
T_{iter}	Number of iterations (in Universal Transformers)
d	Hidden dimension (model width)
N	Number of model parameters
D	Size of training dataset
C	Amount of compute used for training
n	Number of warmup iterations (no gradients)
k	Number of trainable iterations (with gradients)
t	Iteration index (recurrent iterations)
i	Core block index: $i \in \{1, 2, 3\}$
j	Sandwich block index: $j \in \{1, 2, 3, 4\}$
α	Scaling law exponent: $\alpha \approx 0.076$
σ_{noise}^2	Noise scale parameter (hyperparameter)
L	Number of transformer layers (in general transformer architectures)
r	Number of recurrent iterations (used in benchmark comparisons)
ϵ	Numerical stability constant: $\epsilon = 10^{-6}$

Vectors and Tensors

Table 2: Vector and tensor notation

Symbol	Description
<i>Model Inputs and Outputs</i>	

(continued on next page)

(continued from previous page)

Symbol	Description
$\mathbf{x} \in \mathbb{V}^{T_{\text{seq}}}$	Input token sequence (integer token IDs)
$\mathbf{y} \in \mathbb{R}^{T_{\text{seq}} \times \mathbb{V} }$	Output logits over vocabulary
$\mathbf{y}^* \in \mathbb{V}^{T_{\text{seq}}}$	Target token sequence (ground truth)
<i>Hidden States</i>	
$\mathbf{h}_0 \in \mathbb{R}^{T_{\text{seq}} \times d}$	Token embeddings (after embedding layer)
$\mathbf{h}_1 \in \mathbb{R}^{T_{\text{seq}} \times d}$	Hidden state after Prelude layers
$\mathbf{h}_2 \in \mathbb{R}^{T_{\text{seq}} \times d}$	Hidden state after Core blocks
$\mathbf{h}_3 \in \mathbb{R}^{T_{\text{seq}} \times d}$	Hidden state after Coda layers
$\mathbf{h}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Output after i -th block's LayerNorm
$\mathbf{h}_{\text{norm}}^{(0)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Normalized input to first core block
$\tilde{\mathbf{h}}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Output of i -th recurrent block (before residual)
$\mathbf{r}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Output after i -th residual addition
<i>Recurrent Block States</i>	
$\mathbf{x}_{\text{base}} \in \mathbb{R}^{T_{\text{seq}} \times d}$	LayerNorm'd input (fixed across iterations)
$\mathbf{x}_{\text{latent}}^{(t)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Recurrent latent state at iteration t
$\tilde{\mathbf{h}}_t \in \mathbb{R}^{T_{\text{seq}} \times d}$	Combined state after linear mixing
$\mathbf{z}_t^{(j)} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Output of j -th sandwich block at iteration t
<i>Sandwich Block States</i>	
$\mathbf{x}_{\text{attn}} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Intermediate state after attention path
$\mathbf{a} \in \mathbb{R}^{T_{\text{seq}} \times d}$	Attention output
$\mathbf{m} \in \mathbb{R}^{T_{\text{seq}} \times d}$	MLP output
$\hat{\mathbf{x}}, \hat{\mathbf{a}}, \hat{\mathbf{m}}$	Normalized versions of respective tensors
$\hat{\mathbf{x}}_1, \hat{\mathbf{x}}_2 \in \mathbb{R}^{T_{\text{seq}} \times d}$	Normalized intermediate states in sandwich block
<i>Weight Matrices</i>	
$\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times \mathbb{V} }$	Output projection matrix
$\mathbf{W}_{\text{mix}} \in \mathbb{R}^{d \times 2d}$	Linear mixing layer weights
$\mathbf{b}_{\text{mix}} \in \mathbb{R}^d$	Linear mixing layer bias
$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$	Attention projection matrices
$\mathbf{W}_1, \mathbf{W}_2$	MLP weight matrices
<i>Normalization Parameters</i>	
$\gamma \in \mathbb{R}^d$	Scale parameter (RMSNorm/LayerNorm)
$\beta \in \mathbb{R}^d$	Shift parameter (LayerNorm only)
μ	Mean (LayerNorm, computed from data)
σ^2	Variance (LayerNorm, computed from data)
<i>Other</i>	
$\epsilon_t \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{noise}}^2 \mathbf{I}_d)$	Noise vector
$\mathbf{I}_d \in \mathbb{R}^{d \times d}$	Identity matrix
$\mathbf{J}_i$	Jacobian matrix of i -th transformation

Parameter Collections

Table 3: Parameter collection notation

Symbol	Description
θ_{pre}	Prelude layer parameters
θ_{core}	Core block parameters
θ_{coda}	Coda layer parameters
θ_j	Parameters of j -th sandwich block
Θ_i	All parameters for i -th core block
$\Theta = \{\theta_1, \dots, \theta_4\}$	Collection of sandwich block parameters

Sets and Spaces

Table 4: Sets and spaces

Symbol	Description
$\mathbb{V}$	Vocabulary set: $ \mathbb{V} \approx 50,000$ tokens
$\mathbb{N}$	Natural numbers
$\mathbb{R}$	Real numbers
$\mathbb{R}^+$	Positive real numbers
$\mathbb{R}^{T_{\text{seq}} \times d}$	Real matrix space (T_{seq} rows, d columns)
$\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$	Normal distribution

Functions and Operators

Table 5: Functions and operators

Symbol	Description
<i>Model Components</i>	
$\text{Embed}(\cdot)$	Token embedding layer
$\text{Prelude}(\cdot; \theta_{\text{pre}})$	Initial processing (2 layers)
$\text{Core}(\cdot; \theta_{\text{core}})$	Recurrent core blocks (3 blocks)
$\text{Coda}(\cdot; \theta_{\text{coda}})$	Final processing (2 layers)
$\text{LMHead}(\cdot; \mathbf{W}_{\text{out}})$	Language modeling head
$\text{Block}_i(\cdot; \Theta_i)$	i -th recurrent core block
$\text{SandwichBlock}_j(\cdot; \theta_j)$	j -th sandwich block
$f_{\text{core}}(\cdot)$	Composite function of 4 sandwich blocks in sequence
$F(\cdot)$	Generic function placeholder
<i>Normalization</i>	
$\text{LayerNorm}(\cdot; \beta, \gamma)$	Layer Normalization
$\text{RMSNorm}(\cdot; \gamma)$	Root Mean Square Normalization
$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} + \epsilon$	Root mean square
$\text{Norm}(\cdot)$	Generic normalization function
LN_i	Short for $\text{LayerNorm}(\cdot; \beta_i, \gamma_i)$

Table 5: Functions and operators (continued)

Symbol	Description
<i>Attention and MLP</i>	
$\text{Attn}(\cdot; \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O)$	Multi-head self-attention
$\text{MLP}(\cdot; \mathbf{W}_1, \mathbf{W}_2)$	Multi-layer perceptron
<i>Loss and Optimization</i>	
$\mathcal{L} : \mathbb{R}^{T_{\text{seq}} \times \mathbb{V} } \times \mathbb{V}^{T_{\text{seq}}} \rightarrow \mathbb{R}$	Loss function (typically cross-entropy)
$\text{sg}(\cdot)$	Stop-gradient operator
<i>Mathematical Operators</i>	
∇	Gradient operator
$\frac{\partial}{\partial}$	Partial derivative
$\odot$	Element-wise multiplication (Hadamard product)
$\circ$	Function composition
$\top$	Matrix transpose
$\mathbb{E}[\cdot]$	Expectation operator (statistical mean)
$\text{Var}[\cdot]$	Variance operator

Important Distinctions

- **Sequence length vs iterations:** T_{seq} refers to the number of tokens in the input sequence, T_{total} and T_{iter} refer to iteration counts in recurrent models.
- **Loss notation:** $\mathcal{L}$ (calligraphic) is used for the loss function.
- **Block indices:** $i \in \{1, 2, 3\}$ indexes core blocks, $j \in \{1, 2, 3, 4\}$ indexes sandwich blocks inside a recurrent block.
- **Variance notation:** σ^2 is computed variance in LayerNorm (scalar, computed from data), σ_{noise}^2 is a noise scale hyperparameter.
- **Bold notation:** Bold symbols ($\mathbf{x}$, $\mathbf{W}$) are vectors and matrices, non-bold symbols (t , i , d) are scalars.

2.2 TRANSFORMER EVOLUTION

The transformer architecture, introduced by Vaswani et al. (2017) in 2017, revolutionized natural language processing by replacing recurrent connections with self-attention (Figure 2). This architecture enabled parallel processing during training while maintaining the ability to model long range dependencies in sequences. The original design included both encoder and decoder stacks, but following research (Radford et al., 2018) demonstrated that the simpler decoder only architecture can achieve strong performance on language modeling tasks.

The evolution of generative pretrained transformer (GPT) models shows the scaling of modern language models. GPT-1 with 117M parameters showed that unsupervised pretraining on large text corpora, followed by supervised finetuning, could achieve strong performance across a variety of natural language processing (NLP) tasks (Radford et al., 2018). GPT-2 increased the parameter count to 1.5B and showed that language models could perform zero-shot tasks without fine-tuning, introducing the concept of task-agnostic or general-purpose models (Radford et al., 2019). GPT-3 scaled the model size further up to 175B parameters and 96 layers, showing that increasing model size alone could unlock few-shot capabilities where models can adapt to new, before unseen tasks with only a few examples during inference (Brown et al., 2020).

This progression showed the pattern that larger models with more layers and parameters consistently achieved better performance (Kaplan et al., 2020). However, this scaling approach has fundamental downsides. Each additional layer adds more parameters, and achieving deeper models requires

proportionally more memory and training time. The connection between model depth (number of layers) and parameter count means that increasing computational capacity of the model results in larger model size, leading to the resource requirements that currently limit state of the art (SOTA) model development to well funded organizations.

The standard transformer design processes each input exactly once through its sequential layers. For a model with L layers, each token passes through exactly L transformer blocks in a single forward pass. This fixed depth computation limits the model’s ability to allocate more processing power / time to difficult inputs while using less for simple ones. While following work introduced adaptive computation time for RNNs (Graves, 2017), standard transformers lack this flexibility and treat all inputs equally, regardless of complexity of the given task.

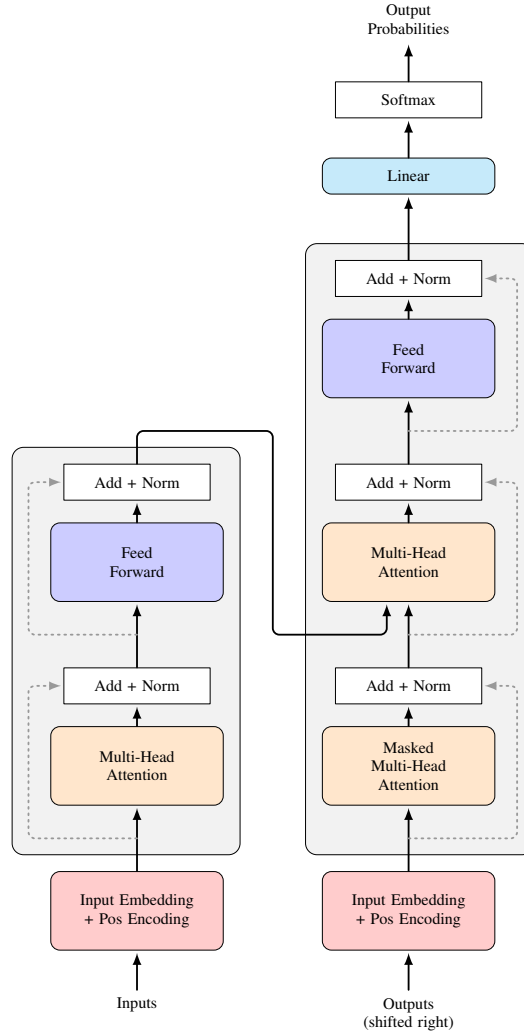


Figure 2: The encoder-decoder transformer architecture, proposed by Vaswani et al. (2017).

2.3 SCALING LAWS

Empirical studies of transformer scaling behaviour have revealed the relationships between model performance and training resources. Kaplan et al. (2020) shows that model performance scales predictably with three factors: the number of parameters (N), the size of the training dataset (D), and the amount of compute (C) used for training (Figure 3). These scaling laws suggest that performance is proportional to the number of parameters:

$$\mathcal{L}(N) \propto N^{-\alpha} \quad (1)$$

where $\mathcal{L}$ is the validation loss, N is the number of parameters, and $\alpha \approx 0.076$. These scaling laws hold across several orders of magnitude and provide a reference framework for model performance before expensive training runs.

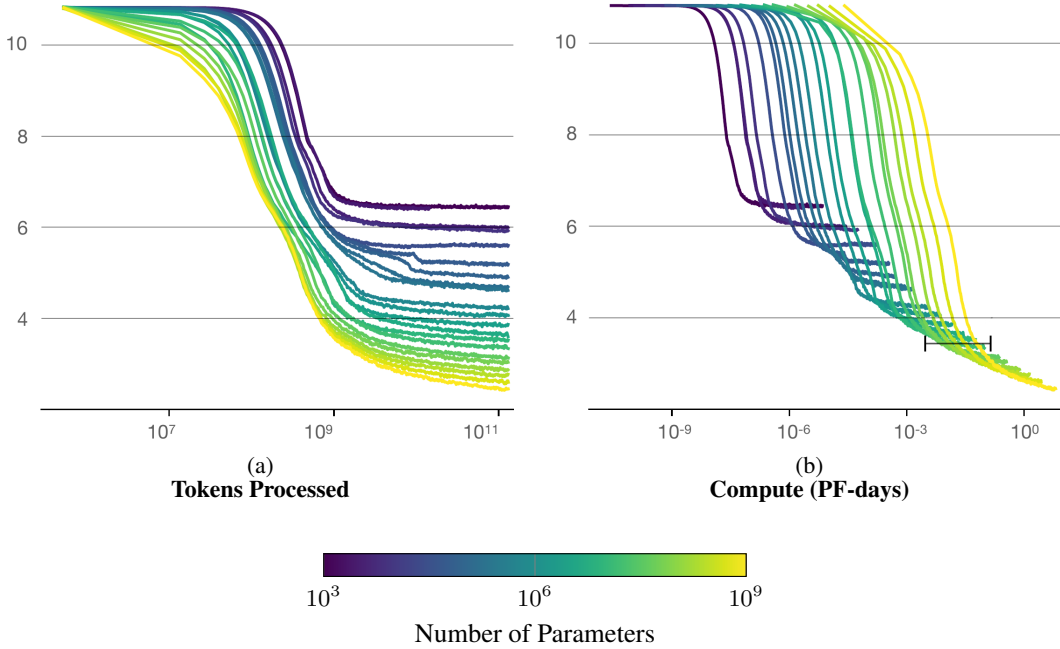


Figure 3: Validation loss against processed tokens and compute. (a) Larger models require fewer samples to reach the same performance as smaller models. (b) Compute-optimal model size grows smoothly with loss target and compute budget. For a target validation loss of 3.4 for example, the compute-optimal model size would be the one that intersects with the marker in the figure at the lowest x value. For a low enough target loss, this model seems to reach the target far before convergence. Model scaling laws, proposed by Kaplan et al. (2020).

It is later shown by Hoffmann et al. (2022) that many large language models are significantly under-trained for their size. Their Chinchilla model achieved better performance than models by training a 70B parameter model on more tokens, rather than scaling the number of parameters with less training data. This work established new scaling laws and suggest that both parameters and training data should scale proportionally (Figures 4 and 5).

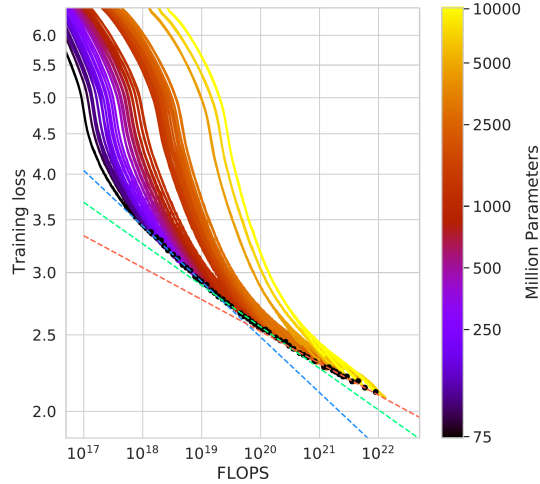


Figure 4: Model scaling laws, proposed by Hoffmann et al. (2022).

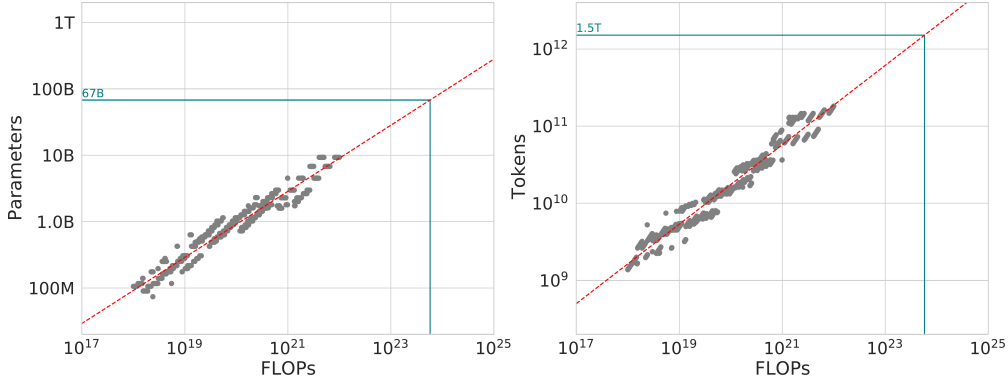


Figure 5: Optimal parameter and training token count for a given compute budget, proposed by Hoffmann et al. (2022).

2.4 PARAMETER SHARING AND RECURRENT APPROACHES

Parameter sharing is an alternative to scaling model depth by adding more layers. Instead of using separate parameters for each layer, parameter sharing reuses the same weights multiple times in different layers, resulting in deeper models without increasing model size. All the models in this section have the same goal: increase the amount of compute per token without proportionally increasing the number of unique parameters. They do so by reusing a set of weights in some way, either along depth, reapplying the same layer, or along time, propagating a hidden state. All following model types are iteratively refining a latent tensor with the same transformation.

Universal Transformers. Dehghani et al. (2019) introduced the Universal Transformer, which applies the same transformer layer multiple times to a latent tensor. Unlike standard transformers with separate parameters for each layer, Universal Transformers use a single transformer layer applied T_{iter} times, where T_{iter} can be dynamically determined with an adaptive computation time mechanism. This architecture is similar to a recurrent neural network, but iterating in depth instead of time, allowing the model to perform more computation on difficult examples while quickly processing simple inputs. While theoretically powerful (Universal Transformers are turing complete with enough recurrent steps), these models have been adopted only limited at scale, possibly due to training instabilities.

ALBERT. Lan et al. (2020) demonstrated that parameter sharing could work effectively even without recurrence by sharing parameters across different layer blocks. ALBERT (A Lite BERT) uses cross layer parameter sharing and factorized embedding parametrization to reduce BERT’s parameter count by up to 18x while maintaining approximately 89% of BERT-base’s performance. Compared to recurrent approaches, ALBERT still processes each shared layer only once per forward pass, so the computational depth remains tied to the number of layers.

Alternatives to Attention. Recent work has explored other architectures that use recurrence or mechanisms with linear time complexity:

Mamba (Gu & Dao, 2024) uses structured state space models (SSMs) to achieve linear time complexity in sequence length while maintaining strong performance. SSMs combine the benefits of CNNs (parallelizable training) and RNNs (efficient autoregressive inference).

RWKV (Peng et al., 2023a) is a hybrid architecture that combines the training efficiency of transformers with the inference efficiency of RNNs, which uses a linear attention mechanism that allows both efficient parallel training and recurrent inference. Like SSMs, RWKV achieves linear complexity in sequence length.

These architectures show that alternatives to transformer architecture can reach competitive performance with higher efficiency and other computational tradeoffs. However, they focus on sequence length efficiency rather than scaling test-time compute through depth recurrence.

Weight Tying in Trained Models. Recent work has explored retrofitting parameter sharing into trained models. Hay & Wolf (2024) use a reinforcement learning algorithm during training to dynamically tie the weights of different transformer layers together, resulting in minimal performance degradation while reducing memory requirements during training. This suggests that intermediate layers learn similar transformations and supports the hypothesis that depth recurrence can be an effective approach to increasing model capacity without increasing parameter count.

2.5 NORMALIZATION TECHNIQUES FOR DEEP NETWORKS

Normalization layers are critical in the training of deep neural networks by stabilizing gradient flow and enabling higher learning rates. For transformer architectures, Layer Normalization (Ba et al., 2016) has become the standard, normalizing the feature dimension of each sample independently.

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mathbb{E}[\mathbf{x}]}{\sqrt{\text{Var}[\mathbf{x}] + \epsilon}} \cdot \gamma + \beta \quad (2)$$

where γ and β are learned scale and shift parameters. Different to Batch Normalization (Ioffe & Szegedy, 2015), Layer Normalization operates independently on each sample, making it suitable for sequences of variable length.

Zhang & Sennrich (2019) introduced Root Mean Square Layer Normalization (RMSNorm), a simplified variant that does not calculate the mean of the values to center the distribution around 0.0:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}} \cdot \gamma \quad (3)$$

RMSNorm reduces computational cost and maintains comparable performance and has been adopted in recent models including LLaMA (Touvron et al., 2023) and PaLM (Chowdhery et al., 2023). The sandwich blocks used in the base architecture by Geiping et al. (2025), which stay a core part of the modified architecture in this paper, employ RMSNorm.

Pre-norm vs. Post-norm. The position of normalization layers relative to residual connections significantly affects training stability and model performance. Pre-norm architectures use normalization layers before the residual connection ($x + F(\text{Norm}(x))$), post-norm architectures normalize after the residual connection ($\text{Norm}(x + F(x))$). These are depicted side by side in Figure 6. Xiong et al. (2020) shows that pre-norm architectures improve gradient flow and enable training of deeper models without learning rate warmup, but the model may achieve slightly worse performance than a well tuned post-norm model. Most modern large language models use pre-norm architectures for training stability.

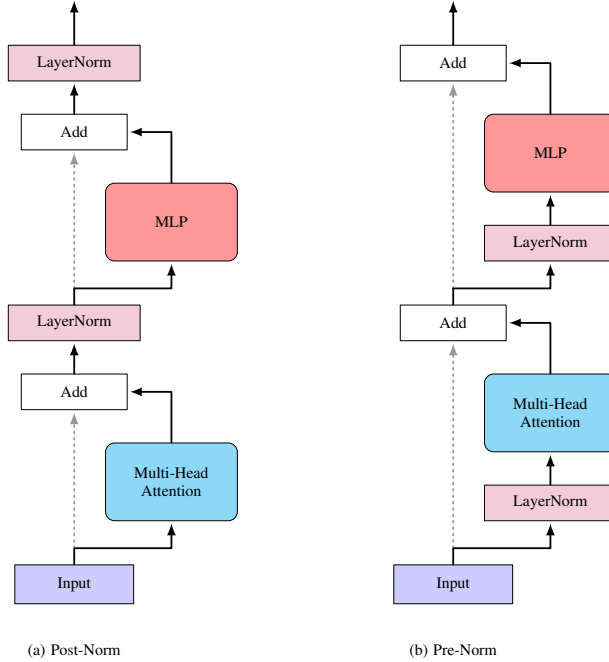


Figure 6: Post-norm and pre-norm architecture, as shown by Xiong et al. (2020).

Normalization in Recurrent Architectures. For recurrent architectures, normalization is even more important. In our architecture with multiple recurrent blocks, we use Layer Normalization between sequential recurrent blocks to stabilize the propagation of activations and gradient flow between blocks. Similarly to deep feedforward networks, this setup helps prevent activations from degrading as information flows through many sequential layers. The sandwich blocks within each recurrent block use RMSNorm following Geiping et al. (2025), and Layer Normalization between recurrent blocks provides additional stabilization.

2.6 TRUNCATED BACKPROPAGATION THROUGH TIME

Backpropagation through time (BPTT) was introduced by Werbos (1990) and extends the backpropagation algorithm to recurrent architectures by unrolling the recurrent computation graph over all recursive iterations. In the case of classical RNNs, these iterations happen over time, across the input sequence, whereas in our case it is an iteration over the latent tensor, so over depth. Gradients are computed through the entire unrolled sequence. As the recurrence depth increases, BPTT faces two critical challenges: memory requirements grow linearly with unroll length as all intermediate activations must be stored, and gradient degradation can occur through long chains of recurrent steps.

Truncated BPTT addresses this by limiting gradient computation to the last k recurrent steps of the total T_{total} recurrent iterations. This offers the advantage of reduced memory usage, $O(k)$ instead of $O(T_{\text{total}})$, and training becomes possible for an arbitrary number of recurrent iterations. Williams & Peng (1990) and Mikolov et al. (2011) demonstrate that truncated BPTT enables training of recurrent models where full BPTT would not be possible due to computational constraints, such as memory

limitations. In depth-recurrent transformers, truncated BPTT behaves differently from temporal RNNs. In RNNs, truncation affects the model’s ability to learn long-range temporal dependencies in a sequence of input data. In depth-recurrent models like Geiping et al. (2025), truncation limits the model’s ability to coordinate gradients across many iterative refinement steps of the latent tensor.

An important property of truncated BPTT in the depth-recurrent model, as proposed by Geiping et al. (2025), is that the prelude layers, which embed inputs into latent space, still receive gradient signals at every iteration because the embedded representation $e = P(x)$ is injected at each recurrent step. This allows the input embedding to be optimized even when gradients don’t flow back through all recurrent iterations. In our multi-block architecture, we apply truncated BPTT independently within each recurrent block, backpropagating through the last k iterations of each recurrent block.

2.7 RECURRENT DEPTH TRANSFORMER

In their work, Geiping et al. (2025) focus on scaling up test-time compute. Inspired by chain-of-thought (Wei et al., 2022) reasoning and iterative refinement of model outputs, they design a novel architecture that enables their LLM to scale compute at test-time without directly increasing output tokens, and therefore high context length requirements.

In their architecture, specifically the final 3.5B parameter model, they use a 2-4-2 layer configuration: 2 prelude layers for embedding input into latent space, 4 recurrent transformer layers in the recurrent block that iterate multiple times over the latent tensor, and 2 coda layers for decoding back to vocabulary space. The model is trained with an average of 32 recurrent steps. The number of recurrent steps is sampled from a log-normal Poisson distribution during training to ensure that the model can generalize to different numbers of recurrent steps at test-time. Additionally, to reduce memory requirements during training, truncated backpropagation, based on truncated backpropagation through time (Mikolov et al., 2011), is employed by only calculating the gradient for the last k iterations of the recurrent block. During their training, k was set to an average of 8. Table 6 compares the performance of this model after training it on 0.8T tokens on the following benchmarks. (Geiping et al., 2025)

- ARC-E: Reasoning, ‘Easy’ set (Clark et al., 2018)
- ARC-C: Reasoning, ‘Challenge’ set (Clark et al., 2018)
- HellaSwag: Common sense text completion (Zellers et al., 2019)
- MMLU: Multitask language understanding (Hendrycks et al., 2021)
- OBQA: Question answering with given relevant information (‘open book’) (Mihaylov et al., 2018)
- PIQA: Common sense questions about physical interactions (Bisk et al., 2020)
- SciQ: Multiple choice science questions (Welbl et al., 2017)
- WinoGrande: Common sense reasoning with pronoun resolution (Sakaguchi et al., 2021)

With increased recurrent steps and therefore higher test-time compute, as shown in the benchmark scores in Table 6, the model achieves higher scores than with fewer recurrences and achieves comparable results to select models with more parameters. Benchmarks that require a lot of real world knowledge, such as MMLU, benefit less from the novel architecture. This might be due to the total number of weights being the limiting factor for knowledge saturation during training, while the increased test-time compute makes up for this on reasoning and logic based benchmarks.

However, while showing solid results in these benchmarks, the single recurrent block architecture has several limitations. First, the long recurrent chain of the transformer layers, repeated up to 64 times during inference, may suffer from gradient flow issues and be prone to a collapsed hidden state despite residual connections, as mentioned in their paper. While truncated backpropagation through time - which is actually more like truncated backpropagation through depth here - addresses memory concerns, calculating the gradient for only the last k steps limits the ability to learn long-range dependencies across the full recurrent depth. Second, the monolithic single block design lacks hierarchical processing stages, applying the same transformer layers in the recurrent block during all recurrent steps. Third, the architecture has limited flexibility at inference time. The number of

Table 6: Model performance comparison across various benchmarks between Geiping et al. (2025)’s original recurrent transformer architecture named Raven, and other models of similar or greater size. Values taken from their paper.

Model	Params	Tokens	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PIQA	SciQ	WinoGrande
random	–	–	25.0	25.0	25.0	25.0	25.0	50.0	25.0	50.0
Amber	7B	1.2T	65.70	37.20	72.54	26.77	41.00	78.73	88.50	63.22
Pythia-2.8b	2.8B	0.3T	58.00	32.51	59.17	25.05	35.40	73.29	83.60	57.85
Pythia-6.9b	6.9B	0.3T	60.48	34.64	63.32	25.74	37.20	75.79	82.90	61.40
Pythia-12b	12B	0.3T	63.22	34.64	66.72	24.01	35.40	75.84	84.40	63.06
OLMo-1B	1B	3T	57.28	30.72	63.00	24.33	36.40	75.24	78.70	59.19
OLMo-7B	7B	2.5T	68.81	40.27	75.52	28.39	42.20	80.03	88.50	67.09
OLMo-7B-0424	7B	2.05T	75.13	45.05	77.24	47.46	41.60	80.09	96.00	68.19
OLMo-7B-0724	7B	2.75T	74.28	43.43	77.76	50.18	41.60	80.69	95.70	67.17
OLMo-2-1124	7B	4T	82.79	57.42	80.50	60.56	46.20	81.18	96.40	74.74
Raven, ($r = 4$)	3.5B	0.8T	49.07	27.99	43.46	23.39	28.20	64.96	80.00	55.24
Raven, ($r = 8$)	3.5B	0.8T	65.11	35.15	58.54	25.29	35.40	73.45	92.10	55.64
Raven, ($r = 16$)	3.5B	0.8T	69.49	37.71	64.67	31.25	37.60	75.79	93.90	57.77
Raven, ($r = 32$)	3.5B	0.8T	69.91	38.23	65.21	31.38	38.80	76.22	93.50	59.43

recurrent iterations is fixed for all layers in the recurrent block during forward propagation, even though the layers might have varying importance during different recurrent steps.

In this work, we try to address these issues by replacing the single recurrent block with multiple smaller, sequentially applied recurrent blocks with intermediate normalization.

3 MULTI-BLOCK RECURRENT TRANSFORMER

The original implementation of the recurrent transformer block architecture is structured around decoder-only transformer blocks, split into three groups: Prelude, Core, and Coda. Each of these groups contains a number of transformer blocks, such as sandwich blocks. The prelude group additionally contains the embedding layer at its start, and the coda group the lm-head at its end. The main part of this novel architecture is the recurrent block, which iterates multiple times over the input tensor. During training, the first n iterations are done without gradient computation, followed by k iterations with gradient computation. (Geiping et al., 2025)

3.1 MOVING FROM ONE BIG RECURRENT BLOCK TO MULTIPLE SMALLER BLOCKS

The proposed architecture is similar to the original implementation, with the difference that instead of a single recurrent core block, multiple recurrent core blocks are applied sequentially in the forward pass. We take a closer look at an example implementation with three recurrent blocks containing four Sandwich Blocks each, as well as two Sandwich Blocks in both the Prelude and the Coda group.

3.2 OVERVIEW

The general model structure is as follows: Token input IDs get embedded in the embedding layer. The embeddings get passed on to the Prelude group, where they get forwarded through two Sandwich Blocks. Following is the Core Blocks group, containing 3 recurrent blocks. After passing through those, the latent tensors pass the two Sandwich Blocks in the Coda group and end in the LM Head, resulting in the output. (Figure 7)

Notation. We define the following notation for the complete RecurrentGPT architecture:

- $\mathbf{x} \in \mathbb{V}^{T_{\text{seq}}}$: input token sequence (integer token IDs)
- $T_{\text{seq}} \in \mathbb{N}$: sequence length (number of tokens)
- $\mathbb{V}$: vocabulary set (typically $|\mathbb{V}| \approx 50,000$ tokens)

- $d \in \mathbb{N}$: hidden dimension (model width)
- $\mathbf{h}_0 \in \mathbb{R}^{T_{\text{seq}} \times d}$: token embeddings (output of embedding layer)
- $\mathbf{h}_1 \in \mathbb{R}^{T_{\text{seq}} \times d}$: hidden state after Prelude layers
- $\mathbf{h}_2 \in \mathbb{R}^{T_{\text{seq}} \times d}$: hidden state after Core recurrent blocks
- $\mathbf{h}_3 \in \mathbb{R}^{T_{\text{seq}} \times d}$: hidden state after Coda layers
- $\mathbf{y} \in \mathbb{R}^{T_{\text{seq}} \times |\mathbb{V}|}$: output logits over vocabulary
- $\mathbf{y}^* \in \mathbb{V}^{T_{\text{seq}}}$: target token sequence (ground truth)
- $\text{Embed}(\cdot)$: token embedding layer mapping tokens to dense vectors
- $\text{Prelude}(\cdot; \boldsymbol{\theta}_{\text{pre}})$: initial processing layers (2 transformer layers)
- $\text{Core}(\cdot; \boldsymbol{\theta}_{\text{core}})$: recurrent core blocks (3 blocks)
- $\text{Coda}(\cdot; \boldsymbol{\theta}_{\text{coda}})$: final processing layers (2 transformer layers)
- $\text{LMHead}(\cdot; \mathbf{W}_{\text{out}})$: language modeling head (linear projection to vocabulary)
- $\boldsymbol{\theta}_{\text{pre}}$: learnable parameters of Prelude layers
- $\boldsymbol{\theta}_{\text{core}}$: learnable parameters of Core blocks
- $\boldsymbol{\theta}_{\text{coda}}$: learnable parameters of Coda layers
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times |\mathbb{V}|}$: output projection matrix
- $\mathcal{L} : \mathbb{R}^{T_{\text{seq}} \times |\mathbb{V}|} \times \mathbb{V}^{T_{\text{seq}}} \rightarrow \mathbb{R}$: loss function (typically cross-entropy)
- $\mathbf{J}_i$: Jacobian matrix of the i -th transformation

Forward pass. The RecurrentGPT model processes inputs through multiple sections, starting with the token embedding layer that maps tokens to embedding vectors. The Prelude stage contains standard transformer layers, specifically in the form of sandwich blocks, that prepare the encoded latent tensor for the following recurrent section. The Core stage contains three recurrent blocks that iteratively process the latent tensor through $n + k$ recurrent steps per block, with the first n iterations being done without gradient calculation, and the following k iterations with gradient calculation. After this, the Coda stage applies additional transformer layers, and the lm-head projects the latent tensor into vocabulary space. The forward pass through the architecture can be expressed as:

$$\mathbf{h}_0 = \text{Embed}(\mathbf{x}) \quad (4)$$

$$\mathbf{h}_1 = \text{Prelude}(\mathbf{h}_0; \boldsymbol{\theta}_{\text{pre}}) \quad (5)$$

$$\mathbf{h}_2 = \text{Core}(\mathbf{h}_1; \boldsymbol{\theta}_{\text{core}}) \quad (6)$$

$$\mathbf{h}_3 = \text{Coda}(\mathbf{h}_2; \boldsymbol{\theta}_{\text{coda}}) \quad (7)$$

$$\mathbf{y} = \text{LMHead}(\mathbf{h}_3; \mathbf{W}_{\text{out}}) \quad (8)$$

Backward pass. The gradient flows backwards through the model using backpropagation, enabling weight updates via gradient descent. Given the loss function $\mathcal{L}(\mathbf{y}, \mathbf{y}^*)$, the gradient of the loss with respect to the output is computed via backpropagation through the chain:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \nabla_{\mathbf{y}} \mathcal{L}(\mathbf{y}, \mathbf{y}^*) \quad (9)$$

The gradients are then propagated backward through each component using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{h}_3} = \nabla_{\mathbf{h}_3} \text{LMHead}(\mathbf{h}_3)^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \quad (10)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \cdot \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} = \nabla_{\mathbf{h}_2} \text{Coda}(\mathbf{h}_2)^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \quad (11)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} \cdot \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} = \nabla_{\mathbf{h}_1} \text{Core}(\mathbf{h}_1)^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} \quad (12)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \cdot \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} = \nabla_{\mathbf{h}_0} \text{Prelude}(\mathbf{h}_0)^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \quad (13)$$

Equivalently, this can be expressed compactly as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_0} = \left(\prod_{i=1}^4 \mathbf{J}_i^\top \right) \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \quad (14)$$

where $\mathbf{J}_i$ denotes the Jacobian of the i -th transformation in the forward pass, ordered from output to input: $\mathbf{J}_1 = \frac{\partial \text{LMHead}}{\partial \mathbf{h}_3}$, $\mathbf{J}_2 = \frac{\partial \text{Coda}}{\partial \mathbf{h}_2}$, $\mathbf{J}_3 = \frac{\partial \text{Core}}{\partial \mathbf{h}_1}$, and $\mathbf{J}_4 = \frac{\partial \text{Prelude}}{\partial \mathbf{h}_0}$.

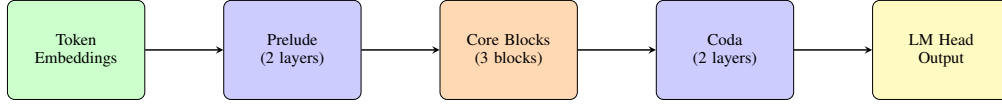


Figure 7: High-level RecurrentGPT architecture overview.

3.3 CORE BLOCKS

The Core Blocks group contains multiple - in this case three - Sandwich Blocks followed by a LayerNorm layer each. This is the main distinction to the original implementation by Geiping et al. (2025), where only a single recurrent block was utilized. To keep the number of trainable parameters approximately the same, we replace a single recurrent block containing 12 Sandwich Blocks with three recurrent blocks containing 4 Sandwich Blocks each. The input tensor passes through each recurrent block and the following LayerNorm sequentially. The choice of specifically 3 recurrent blocks with 4 Sandwich Blocks each is mostly arbitrary: We need at least two recurrent blocks for the architecture to have multiple recurrent blocks, with at least a few Sandwich Blocks in each recurrent block to allow for ‘deep’ enough learning. Although it would be possible to use more than 3 recurrent blocks or more than 4 Sandwich blocks, we would need to decrease the hidden dimension accordingly to keep the total number of model parameters consistent. This would likely negatively affect the model at our small testing scale, which is described later in more detail. (Figure 8)

Notation. We define the following notation for the sequence of three recurrent core blocks:

- $\mathbf{h}^{(0)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: input from the Prelude layers
- $\tilde{\mathbf{h}}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: output of the i -th recurrent core block (before residual addition), for $i \in \{1, 2, 3\}$
- $\mathbf{r}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: output after the i -th residual addition
- $\mathbf{h}^{(i)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: output after the i -th LayerNorm (input to next block), for $i \in \{1, 2, 3\}$
- $\text{Block}_i(\cdot; \boldsymbol{\Theta}_i)$: the i -th recurrent core block containing 4 sandwich blocks
- $\boldsymbol{\Theta}_i$: all parameters for the i -th core block (includes parameters for 4 sandwich blocks)
- $\text{LayerNorm}(\cdot; \beta_i, \gamma_i)$: Layer Normalization with learnable shift $\beta_i \in \mathbb{R}^d$ and scale $\gamma_i \in \mathbb{R}^d$
- $n \in \mathbb{N}$: number of warmup iterations per block (no gradients)
- $k \in \mathbb{N}$: number of trainable iterations per block (with gradients)

Forward pass. Each of the core blocks contains a pre-normalization residual architecture with accumulating residual connections. Every block does the following: pre-normalization of input, $n + k$ recurrent iterations, addition of the output to the residual connection. This creates direct gradient paths from output to input and each transformer block. The final accumulated residual $\mathbf{r}^{(3)}$ is passed directly to the Coda block.

The forward pass is described as follows:

$$\mathbf{h}_{\text{norm}}^{(0)} = \text{LayerNorm}(\mathbf{h}^{(0)}; \beta_0, \gamma_0) \quad (15)$$

$$\tilde{\mathbf{h}}^{(1)} = \text{Block}_1(\mathbf{h}_{\text{norm}}^{(0)}; \Theta_1) \quad (n + k \text{ iterations}) \quad (16)$$

$$\mathbf{r}^{(1)} = \mathbf{h}^{(0)} + \tilde{\mathbf{h}}^{(1)} \quad (17)$$

$$\mathbf{h}^{(1)} = \text{LayerNorm}(\mathbf{r}^{(1)}; \beta_1, \gamma_1) \quad (18)$$

$$\tilde{\mathbf{h}}^{(2)} = \text{Block}_2(\mathbf{h}^{(1)}; \Theta_2) \quad (n + k \text{ iterations}) \quad (19)$$

$$\mathbf{r}^{(2)} = \mathbf{r}^{(1)} + \tilde{\mathbf{h}}^{(2)} \quad (20)$$

$$\mathbf{h}^{(2)} = \text{LayerNorm}(\mathbf{r}^{(2)}; \beta_2, \gamma_2) \quad (21)$$

$$\tilde{\mathbf{h}}^{(3)} = \text{Block}_3(\mathbf{h}^{(2)}; \Theta_3) \quad (n + k \text{ iterations}) \quad (22)$$

$$\mathbf{r}^{(3)} = \mathbf{r}^{(2)} + \tilde{\mathbf{h}}^{(3)} \quad (23)$$

The output $\mathbf{r}^{(3)}$ is then passed to the Coda layers. Note that the residual connections accumulate across blocks: $\mathbf{r}^{(i)} = \mathbf{r}^{(i-1)} + \tilde{\mathbf{h}}^{(i)}$ for $i \geq 2$, with $\mathbf{r}^{(1)} = \mathbf{h}^{(0)} + \tilde{\mathbf{h}}^{(1)}$. This can be written as:

$$\mathbf{r}^{(3)} = \mathbf{h}^{(0)} + \sum_{i=1}^3 \tilde{\mathbf{h}}^{(i)} \quad (24)$$

The complete forward pass through all three core blocks can be written as:

$$\mathbf{r}^{(3)} = \mathbf{h}^{(0)} + \sum_{i=1}^3 \text{Block}_i(\text{LN}_{i-1}(\cdot); \Theta_i) \quad (25)$$

where LN_i is the LayerNorm with parameters (β_i, γ_i) , and LN_0 operates on $\mathbf{h}^{(0)}$ and $\text{LN}_{i>0}$ operates on the accumulated residuals.

LayerNorm definition. Layer Normalization normalizes activations across the feature dimension:

$$\text{LayerNorm}(\mathbf{x}; \beta, \gamma) = \gamma \odot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (26)$$

where $\odot$ is element-wise multiplication, and for each position in the sequence:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i \quad (\text{mean}) \quad (27)$$

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad (\text{variance}) \quad (28)$$

with $\epsilon \in \mathbb{R}^+$ being a small constant for numerical stability ($\epsilon = 10^{-6}$).

Backward pass. The gradients for each block get calculated through both the residual connection path and the computational path of transformer blocks and normalization layers. Gradients split at each residual connection and sum after flowing through both paths. With this architecture, each block receives signals from all subsequent blocks. In each block, the gradient calculation is limited to the last k recurrent steps by truncated backpropagation through time.

The gradient flows backward through the three blocks sequentially. Starting from the loss gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(3)}}$:

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(3)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(3)}} \quad (\text{residual branch}) \quad (29)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(3)}} \cdot \nabla_{\mathbf{h}^{(2)}} \text{Block}_3(\mathbf{h}^{(2)})^\top \quad (30)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(2)}} \cdot \nabla_{\mathbf{r}^{(2)}} \text{LayerNorm}(\mathbf{r}^{(2)})^\top + \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(3)}} \quad (\text{residual skip}) \quad (31)$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(2)}} \quad (\text{residual branch}) \quad (32)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(2)}} \cdot \nabla_{\mathbf{h}^{(1)}} \text{Block}_2(\mathbf{h}^{(1)})^\top \quad (33)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \cdot \nabla_{\mathbf{r}^{(1)}} \text{LayerNorm}(\mathbf{r}^{(1)})^\top + \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(2)}} \quad (\text{residual skip}) \quad (34)$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(1)}} \quad (\text{residual branch}) \quad (35)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{\text{norm}}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}^{(1)}} \cdot \nabla_{\mathbf{h}_{\text{norm}}^{(0)}} \text{Block}_1(\mathbf{h}_{\text{norm}}^{(0)})^\top \quad (36)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{\text{norm}}^{(0)}} \cdot \nabla_{\mathbf{h}^{(0)}} \text{LayerNorm}(\mathbf{h}^{(0)})^\top + \frac{\partial \mathcal{L}}{\partial \mathbf{r}^{(1)}} \quad (\text{residual skip}) \quad (37)$$

Note that within each Block_i , gradients are computed using truncated backpropagation through time for only the last k of the $n + k$ recurrent iterations. The residual connections provide direct gradient paths that bypass the recurrent blocks, while the normalization layers help stabilize gradient flow through the three sequential blocks.

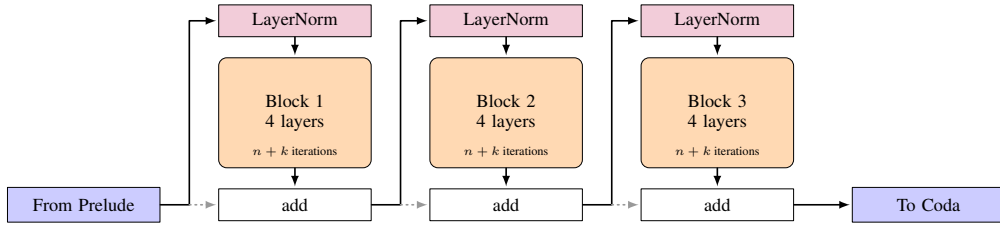


Figure 8: Core recurrent blocks with per-block normalization and residual connections.

3.4 RECURRENT BLOCK

Inside the recurrent block, a scaled tensor containing normal distributed noise gets added to the input tensor. The noise tensor is defined as $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{noise}}^2 \mathbf{I}_d)$ where σ_{noise}^2 is the noise scale parameter and $\mathbf{I}_d$ is the $d \times d$ identity matrix. The sequence of Sandwich Blocks then iterates $n + k$ times over the latent tensor, with the gradient only being computed during the last k iterations. (Figure 9)

Notation. We define the following notation for the recurrent core block:

- $\mathbf{x}_{\text{base}} \in \mathbb{R}^{T_{\text{seq}} \times d}$: the LayerNorm'd input tensor (fixed across all iterations)
- $\mathbf{x}_{\text{latent}}^{(t)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: recurrent latent state at iteration t , where T_{seq} is sequence length and d is hidden dimension
- $\tilde{\mathbf{h}}_t \in \mathbb{R}^{T_{\text{seq}} \times d}$: combined state after linear mixing (input to sandwich blocks)
- $\mathbf{W}_{\text{mix}} \in \mathbb{R}^{d \times 2d}$, $\mathbf{b}_{\text{mix}} \in \mathbb{R}^d$: learnable parameters of the linear mixing layer
- $\mathbf{z}_t^{(j)} \in \mathbb{R}^{T_{\text{seq}} \times d}$: intermediate output of the j -th sandwich block at iteration t , where $j \in \{1, 2, 3, 4\}$
- θ_j : learnable parameters of the j -th sandwich block
- $n \in \mathbb{N}$: number of recurrent iterations without gradient tracking
- $k \in \mathbb{N}$: number of recurrent iterations with gradient tracking
- $\mathcal{L}$: loss function
- $\text{sg}(\cdot)$: stop-gradient operator (prevents gradient flow)

Forward pass. The Core Block processes the input over $n+k$ recurrent iterations, where gradients are computed only for the last k steps. The latent state is initialized with a normal distribution of random noise and refined through recurrent iterations with learnable linear mixing of the base input and recurrent state. After linearly combining the initialized latent state and the base input into a latent tensor, it gets processed sequentially by the following sandwich blocks. The output of the last sandwich block is used during the next recurrent step in place of the previous latent state. After the last recurrent step, the output of the final sandwich block becomes the output of the recurrent block.

The forward pass starts with the initialization of the latent state:

$$\mathbf{x}_{\text{latent}}^{(0)} \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{noise}}^2 \mathbf{I}_d) \quad (38)$$

where σ_{noise}^2 is the noise scale parameter and $\mathbf{I}_d$ is the $d \times d$ identity matrix.

The forward pass for a single iteration is as follows. At each iteration t , the base input and current latent state are concatenated and mixed through a learnable linear layer:

$$\tilde{\mathbf{h}}_t = \mathbf{W}_{\text{mix}} \begin{bmatrix} \mathbf{x}_{\text{base}} \\ \mathbf{x}_{\text{latent}}^{(t)} \end{bmatrix} + \mathbf{b}_{\text{mix}} \quad (39)$$

$$\mathbf{z}_t^{(1)} = \text{SandwichBlock}_1(\tilde{\mathbf{h}}_t; \theta_1) \quad (40)$$

$$\mathbf{z}_t^{(2)} = \text{SandwichBlock}_2(\mathbf{z}_t^{(1)}; \theta_2) \quad (41)$$

$$\mathbf{z}_t^{(3)} = \text{SandwichBlock}_3(\mathbf{z}_t^{(2)}; \theta_3) \quad (42)$$

$$\mathbf{x}_{\text{latent}}^{(t+1)} = \text{SandwichBlock}_4(\mathbf{z}_t^{(3)}; \theta_4) \quad (43)$$

where $[\mathbf{x}_{\text{base}}; \mathbf{x}_{\text{latent}}^{(t)}] \in \mathbb{R}^{T_{\text{seq}} \times 2d}$ is the concatenation along the feature dimension.

The composition of all four sandwich blocks can be written as:

$$f_{\text{core}}(\mathbf{h}; \Theta) = \text{SandwichBlock}_4 \circ \text{SandwichBlock}_3 \circ \text{SandwichBlock}_2 \circ \text{SandwichBlock}_1(\mathbf{h}) \quad (44)$$

where $\Theta = \{\theta_1, \theta_2, \theta_3, \theta_4\}$ are all sandwich block parameters.

For the first n iterations, computation is performed without gradient calculation:

$$\mathbf{x}_{\text{latent}}^{(t+1)} = \text{sg} \left(f_{\text{core}}(\tilde{\mathbf{h}}_t; \boldsymbol{\Theta}) \right), \quad t \in \{0, 1, \dots, n-1\} \quad (45)$$

For the last k iterations, gradients are calculated:

$$\mathbf{x}_{\text{latent}}^{(t+1)} = f_{\text{core}}(\tilde{\mathbf{h}}_t; \boldsymbol{\Theta}), \quad t \in \{n, n+1, \dots, n+k-1\} \quad (46)$$

The final output after $n+k$ iterations is $\mathbf{x}_{\text{latent}}^{(n+k)}$.

Backward pass. Truncated backpropagation through time limits gradient calculation to the final k iterations. Starting from $\mathbf{x}_{\text{latent}}^{(n+k)}$, the gradient flows backward through the sandwich blocks and linear mixing layer at each iteration. The linear mixing layer gradient splits between base input and recurrent state, with only the recurrent part of the gradient propagating back to the last sandwich block for the last k iterations.

During backpropagation, the gradient at the output $\mathbf{x}_{\text{latent}}^{(n+k)}$ (denoted as $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(n+k)}}$) is propagated backward through time over the last k iterations.

The gradient through the linear mixing operation with respect to the latent state is:

$$\frac{\partial \tilde{\mathbf{h}}_t}{\partial \mathbf{x}_{\text{latent}}^{(t)}} = \mathbf{W}_{\text{mix}}[:, d : 2d] \quad (47)$$

where $\mathbf{W}_{\text{mix}}[:, d : 2d] \in \mathbb{R}^{d \times d}$ is the second half of the weight matrix corresponding to the latent state input.

For each trainable iteration, the gradient with respect to the latent state is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(t)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(t+1)}} \cdot \nabla_{\tilde{\mathbf{h}}_t} f_{\text{core}}(\tilde{\mathbf{h}}_t; \boldsymbol{\Theta})^\top \cdot \mathbf{W}_{\text{mix}}[:, d : 2d]^\top \quad (48)$$

for $t \in \{n, n+1, \dots, n+k-1\}$.

The accumulated gradient through all k trainable iterations is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(n)}} = \left(\prod_{t=n}^{n+k-1} \mathbf{W}_{\text{mix}}[:, d : 2d]^\top \cdot \nabla_{\tilde{\mathbf{h}}_t} f_{\text{core}}(\tilde{\mathbf{h}}_t; \boldsymbol{\Theta}) \right)^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(n+k)}} \quad (49)$$

Due to the stop-gradient operation, gradients do not propagate beyond iteration n :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{latent}}^{(t)}} = \mathbf{0}, \quad \forall t < n \quad (50)$$

The learnable linear mixing layer $\mathbf{W}_{\text{mix}}$ enables the model to control the contribution of the base input and recurrent state at each iteration. This enables more dynamic ratios and weighting of different features than fixed mixing ratios while maintaining stable gradient flow through the recurrent connection.

This truncated backpropagation through time with learnable linear mixing limits computational cost and memory requirements while enabling gradient-based optimization through the last k recurrent steps.

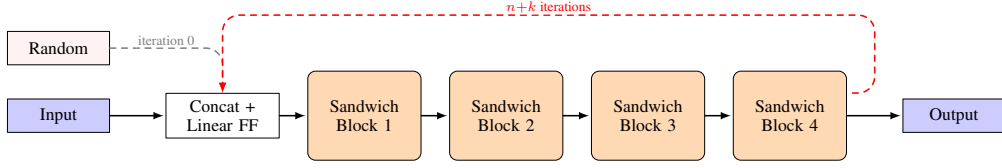


Figure 9: Internals of Recurrent Block. Each block iterates over the latent tensor for $n + k$ steps (n steps without gradients, k steps with gradients), with a scaled input tensor added to it before every step.

3.5 SANDWICH BLOCK

The Sandwich Block consists of two sequential processing paths: an attention path and an MLP path. Each path follows a normalization-operation-normalization pattern, with the operation sandwiched between two RMSNorm layers. The input tensor first passes the attention path, then the MLP path. The output of each path is added to their input tensor via a residual connection to stabilize gradient flow and prevent gradient decay. (Figure 10)

Notation. We define the following notation for a single Sandwich Block:

- $\mathbf{x} \in \mathbb{R}^{T_{\text{seq}} \times d}$: input to the sandwich block
- $\mathbf{y} \in \mathbb{R}^{T_{\text{seq}} \times d}$: output of the sandwich block
- $\mathbf{x}_{\text{attn}} \in \mathbb{R}^{T_{\text{seq}} \times d}$: intermediate state after attention path
- $\text{RMSNorm}(\cdot; \gamma)$: Root Mean Square Layer Normalization with learnable scale $\gamma \in \mathbb{R}^d$
- $\text{Attn}(\cdot; \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O)$: multi-head self-attention with query, key, value, and output projection matrices
- $\text{MLP}(\cdot; \mathbf{W}_1, \mathbf{W}_2)$: multi-layer perceptron (feedforward network) with weight matrices
- $\gamma_1, \gamma_2, \gamma_3, \gamma_4$: scale parameters for the four RMSNorm layers

Forward pass. The sandwich block processes information through the sequential attention path and multi layer perceptron (MLP) path, each normalizing their input before and after the operation with RMSNorm layers. The attention path normalizes the input, applies multi head self-attention, normalizes the output and adds it to the latent tensor via residual connection. The MLP path uses a copy of this latent tensor as input, normalizes it, processes it through a feedforward network, normalizes the output and adds it to the latent tensor via residual connection.

The Sandwich Block applies two sequential sub-blocks with residual connections. First, the attention path processes the input:

$$\hat{\mathbf{x}}_1 = \text{RMSNorm}(\mathbf{x}; \gamma_1) \quad (51)$$

$$\mathbf{a} = \text{Attn}(\hat{\mathbf{x}}_1; \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O) \quad (52)$$

$$\hat{\mathbf{a}} = \text{RMSNorm}(\mathbf{a}; \gamma_2) \quad (53)$$

$$\mathbf{x}_{\text{attn}} = \mathbf{x} + \hat{\mathbf{a}} \quad (54)$$

Then, the MLP path processes the intermediate representation:

$$\hat{\mathbf{x}}_2 = \text{RMSNorm}(\mathbf{x}_{\text{attn}}; \gamma_3) \quad (55)$$

$$\mathbf{m} = \text{MLP}(\hat{\mathbf{x}}_2; \mathbf{W}_1, \mathbf{W}_2) \quad (56)$$

$$\hat{\mathbf{m}} = \text{RMSNorm}(\mathbf{m}; \gamma_4) \quad (57)$$

$$\mathbf{y} = \mathbf{x}_{\text{attn}} + \hat{\mathbf{m}} \quad (58)$$

The complete forward pass can be expressed compactly as:

$$\begin{aligned} \mathbf{y} = & \mathbf{x} + \text{RMSNorm}(\text{Attn}(\text{RMSNorm}(\mathbf{x}))) \\ & + \text{RMSNorm}\left(\text{MLP}\left(\text{RMSNorm}\left(\mathbf{x} + \text{RMSNorm}(\text{Attn}(\text{RMSNorm}(\mathbf{x})))\right)\right)\right) \end{aligned} \quad (59)$$

RMSNorm definition. Unlike standard LayerNorm, RMSNorm does not center values around zero. It is defined as:

$$\text{RMSNorm}(\mathbf{x}; \gamma) = \gamma \odot \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \quad (60)$$

where $\odot$ denotes element-wise multiplication, and the root mean square is computed as:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (61)$$

with $\epsilon \in \mathbb{R}^+$ being a small constant for numerical stability ($\epsilon = 10^{-6}$).

Backward pass. The gradient flows backwards through each residual connection where it splits to both the previous residual connection and the respective processing path of the sandwich block, the MLP or the attention path.

The gradient flows backward through both paths, starting from $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{attn}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} + \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \frac{\partial \hat{\mathbf{m}}}{\partial \mathbf{x}_{\text{attn}}} \quad (62)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{attn}}} + \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{\text{attn}}} \cdot \frac{\partial \hat{\mathbf{a}}}{\partial \mathbf{x}} \quad (63)$$

where the gradients through each sub-block are computed using the chain rule:

$$\frac{\partial \hat{\mathbf{m}}}{\partial \mathbf{x}_{\text{attn}}} = \nabla_{\mathbf{x}_{\text{attn}}} \text{RMSNorm}(\text{MLP}(\text{RMSNorm}(\mathbf{x}_{\text{attn}})))^\top \quad (64)$$

$$\frac{\partial \hat{\mathbf{a}}}{\partial \mathbf{x}} = \nabla_{\mathbf{x}} \text{RMSNorm}(\text{Attn}(\text{RMSNorm}(\mathbf{x})))^\top \quad (65)$$

The dual normalization structure (pre-normalization before the block and post-normalization after) enhances gradient flow by preventing gradient vanishing and stabilizing training dynamics.

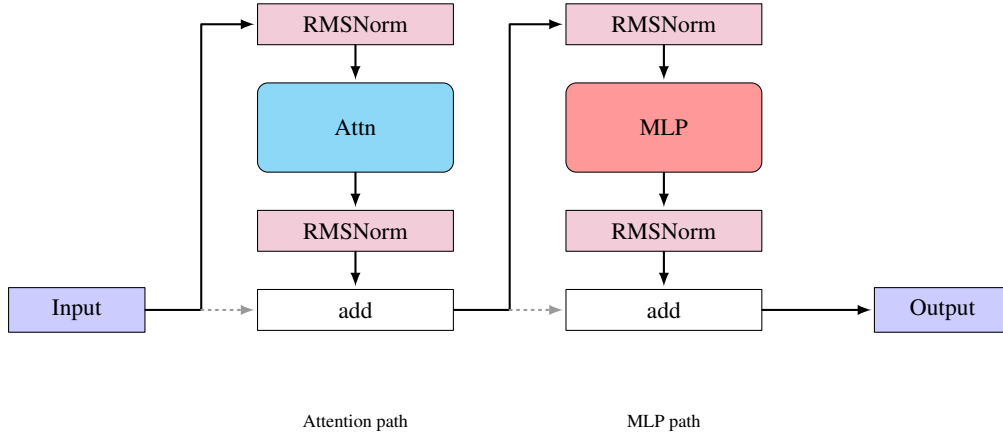


Figure 10: Sandwich block internal structure: Norm-Attn-Norm-Norm-MLP-Norm with residual connections. This provides better gradient flow compared to standard pre-norm blocks. For normalization, RMSNorm is used.

4 EVALUATION

In this section, we first go over the general training setup, then analyze the training history, benchmark the trained model, and finish with example model outputs.

4.1 SETUP

Following is the specific setup of the training and evaluation, with information about hardware and software, hyperparameters, and the dataset used for the training run.

4.1.1 HARDWARE AND SOFTWARE

The hardware used for these experiments was provided by Technische Hochschule Ingolstadt. The final model was trained on 4x A100-80GB over 2.5 days. The model was trained using PyTorch 2.9.0 and PyTorch-Lightning 2.3.0.dev20240328 using lightning’s Distributed Data Parallel (DDP) strategy.

4.1.2 HYPERPARAMETERS

The model was trained using the hyperparameters specified in Table 7. The model has a total number of approximately 308M parameters, 201M of those in the recurrent blocks, which unfold to 2,523M parameters with 12 recursive steps or 4,940M parameters with 24 recursive steps. The model has an embedding dimension of 1024 with a vocabulary size of 32000 using the llama2-tokenizer, an intermediate size of 4096 in the MLP blocks, and 16 attention heads in the attention blocks. The maximum input sequence length during training was truncated to 2048 and the lm-head weights were tied to the embedding weights. During training, the number of recurrent steps T_{total} is sampled separately for each minibatch from a Poisson-lognormal distribution and clamped to a minimum of 1. The Poisson rate is drawn from $\text{LogNormal}(\mu, 0.5)$, calibrated to mean 12 with variance ≈ 38 . The backpropagation depth k is calculated by clamping T_{total} to a maximum of 8.

Table 7: Hyperparameters for all training runs.

Parameter	Value
Model Architecture	
Parameters	$\sim 308\text{M}$
Embedding Dimension	1024

Table 7: Hyperparameters for all training runs. (continued)

Parameter	Value
Intermediate Size (FFN)	4096
Activation Function	SiLU
Attention Heads	16
Vocab Size	32000
Sequence Length	2048
Tie Embeddings	True
Mean Recurrence	12
Max Backprop Depth	8
Optimization	
Batch Size	1024
Learning Rate	$3 \times 10^{-4} - 5 \times 10^{-5}$
Weight Decay	4×10^{-5}
Gradient Clipping	1.0
Precision	Mixed (bfloat16)
Optimizer	ELLISAdam

ELLISAdam, mentioned in Table 7, is an extension of the Adam optimizer (Kingma & Ba, 2014) with additions for decoupled weight decay, weight update modifications and options to enhance numerical stability and precision. Due to this optimizer being used in the original paper by Geiping et al. (2025), we chose to keep using it in our experiments to enable reliable comparison of our architectural modifications and results to the original implementation.

4.1.3 DATASET

Table 8: Data Mixture for Pre-training Phase 1

Category	Dataset	Weight (%)
Web & General		80.0
	fineweb-edu	65.0
	wikipedia	9.0
	books	6.0
Code		12.0
	code-py	3.60
	code-js	2.40
	code-ts	1.20
	code-java	1.20
	code-cpp	0.96
	code-go	0.84
	code-rust	0.60
	code-shell	0.48
	code-sql	0.36
	code-html	0.36
Academic & Math		8.0
	peso	3.0

Table 8: Data Mixture for Pre-training Phase 1 (continued)

Category	Dataset	Weight (%)
	arxiv	2.0
	math	3.0

Table 9: Data Mixture for Pre-training Phase 2

Category	Dataset	Weight (%)
Web Content		35.0
	fineweb-edu	35.0
Code		28.0
	code-py	8.40
	code-js	5.60
	code-ts	2.80
	code-java	2.80
	code-cpp	2.24
	code-go	1.96
	code-rust	1.40
	code-shell	1.12
	code-sql	0.84
	code-html	0.84
Math Mixture		22.0
	math-owm	8.8
	math-gsm	6.6
	math-alg	4.4
	math-gsm8k	2.2
Academic Papers		15.0
	peso	9.0
	arxiv	6.0

Table 10: Data Mixture for Finetuning Stage

Dataset	Weight (%)
FLAN-Focused Balanced Mixture	
FLAN Collection	40.0
MetaMathQA	15.0
Orca-Math	10.0
Evol-Instruct-Code	12.5
Code Alpaca	2.5
SlimOrca-Dedup	10.0
ShareGPT (filtered)	5.0
WizardLM Evol V2	5.0

Training of the final model was done using 3 distinct training phases with different dataset mixes and learning rates (Tables 8 to 10). Due to time and resource constraints, the total amount of tokens trained on was limited to 5 billion tokens.

Phase 1. Phase 1 (Table 8) focuses on broad language learning and knowledge acquisition over 3.3 billion tokens. The dataset mix focuses on general web content 65% FineWeb (Lozhkov et al., 2024), 9% Wikipedia (Foundation), 6% books (sedthh, 2023), in order to establish a foundational understanding of language. 12% of the mixture focuses on code data (codeparrot, 2022), distributed across ten programming languages. Academic content, specifically peS2o (Soldaini & Lo, 2023) and arxiv papers (Kandpal et al., 2025), as well as the math-focused OpenWebMath (Paster et al., 2023) make up 8% of the mix.

Phase 2. Phase 2 (Table 9) focuses on domain-specific upsampling over 1.5 billion tokens, and increases the proportion of code and mathematical content. Code data is upsampled to 28% of the mixture. Mathematical content is upsampled to 22%, using sources like OpenWebMath (8.8%), GSM-style problems (ostapeno, 2025) (6.6%), algebraic-stack from the proof-pile-2 (Azerbaiyev et al., 2023) (4.4%), and GSM8K-train (Cobbe et al., 2021) (2.2%). Academic papers from PESO and arXiv make up 15% of the mixture, general web content from FineWeb is reduced to 35%.

Phase 3. Phase 3 (Table 10) focuses on supervised fine-tuning over 150 million tokens. The main part of this dataset mix consists of the FLAN Collection (Goodson, 2023) (40%), which provides coverage of diverse tasks and formats. MetaMathQA (Yu et al., 2023) (15%) and Orca-Math (Mitra et al., 2024) (10%) are added for mathematical reasoning, Evol-Instruct-Code (nickrosh, 2023) (12.5%) and Code Alpaca (sahil2801, 2023) (2.5%) for coding capabilities. SlimOrca-Dedup (Hoos et al., 2023) (10%), ShareGPT filtered (anon8231489123, 2023) (5%), and WizardLM Evol V2 (WizardLMTeam, 2023) (5%) are added for general instruction-following capabilities.

Phase transitions. For smooth transitions between training phases and to avoid abrupt distribution shifts of the training data, we use a linear interpolation strategy. During the final 10% of samples in each phase, the current phase’s dataset mix ratio linearly decreases from 1.0 to 0.0 while the following phase’s mixture ratio increases from 0.0 to 1.0. This gradual transition is applied between Phase 1 and Phase 2, and between Phase 2 and Phase 3.

4.2 TRAINING

The model was trained with a global batch size of 1024 samples per optimizer step and a minibatch size of 4. As described before, the training was carried out over 3 phases with distinct dataset mixtures for each. Additionally, each phase has a separate learning rate. The training started with a warmup phase of 64 optimizer steps or a total of 16384 samples, increasing the learning rate to 3×10^{-4} for phase 1. During the transition to phase 2 over the last 10% of phase 1, the learning rate linearly decreased to 1×10^{-4} . During the transition from phase 2 to phase 3, the finetuning phase, the learning rate decreased further to 5×10^{-5} . The learning rate curves are visualized in Figure 12.

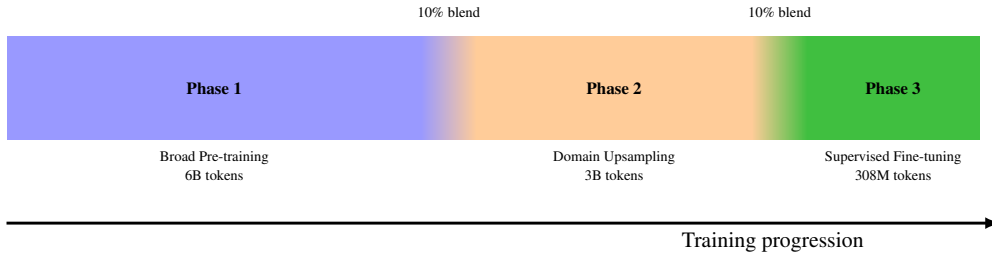


Figure 11: Training phases with smooth transitions. During the final 10% of each phase, the dataset mix linearly interpolates between the current and next phase.

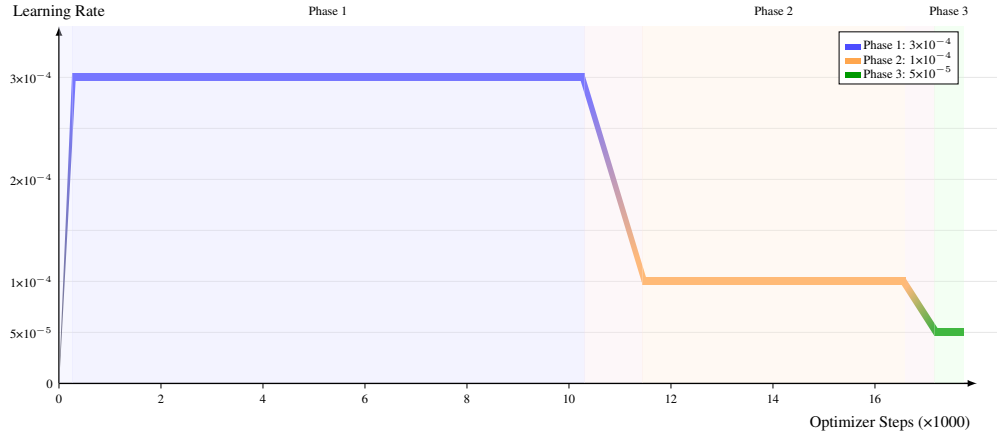


Figure 12: Learning rate schedule across training phases. The learning rate warms up from 0 to 3×10^{-4} during the first 64 optimizer steps, remains constant during each phase, and linearly interpolates learning rates in phase transitions.

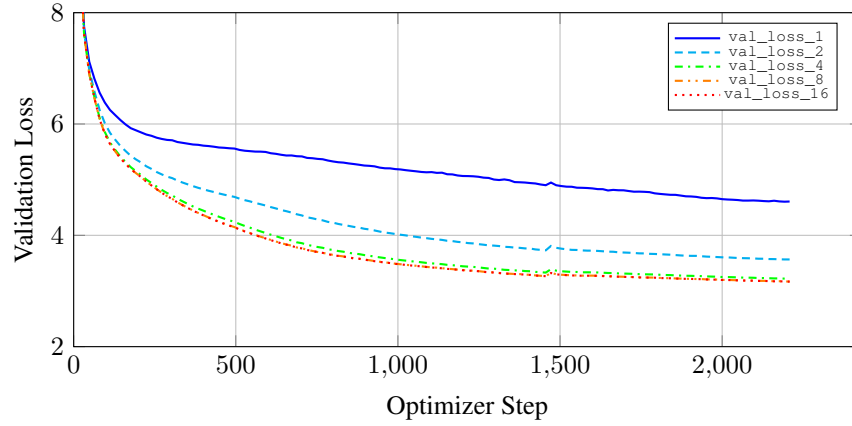


Figure 13: Loss on the validation dataset during pretraining. Each line is the loss for a given number of recurrent steps.

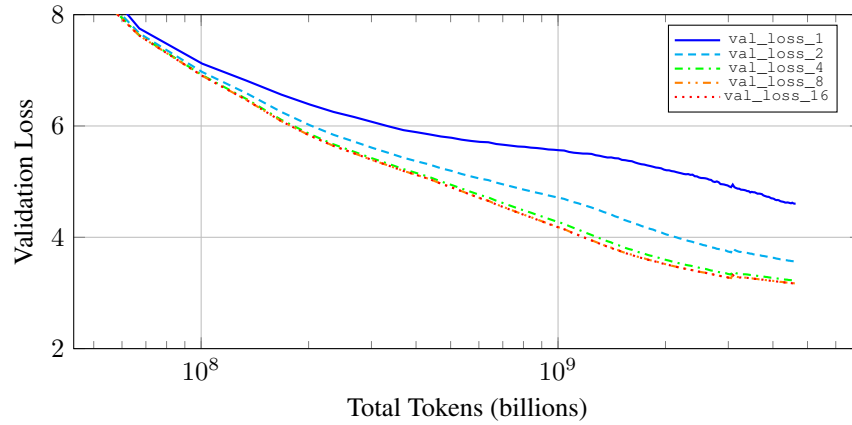


Figure 14: Loss on the validation dataset during pretraining against total tokens trained on.

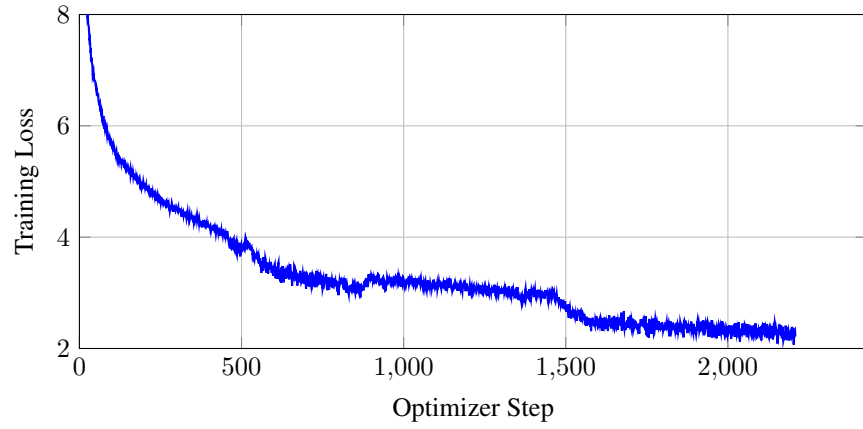


Figure 15: Loss on the training dataset during pretraining.

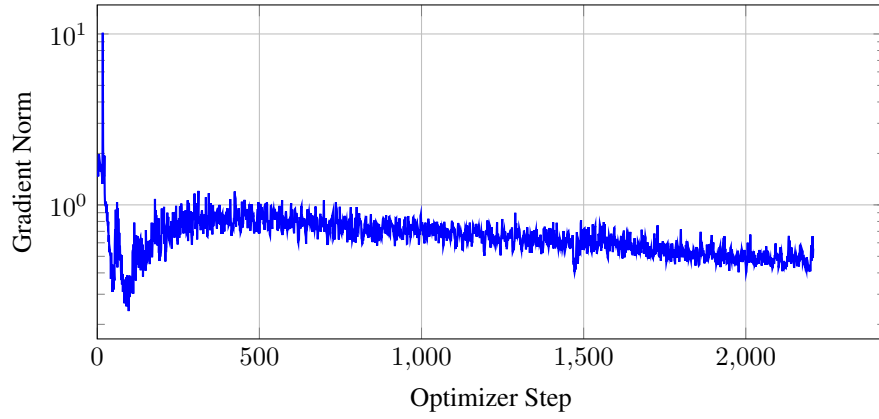


Figure 16: Gradient Norm of trained weights during pretraining.

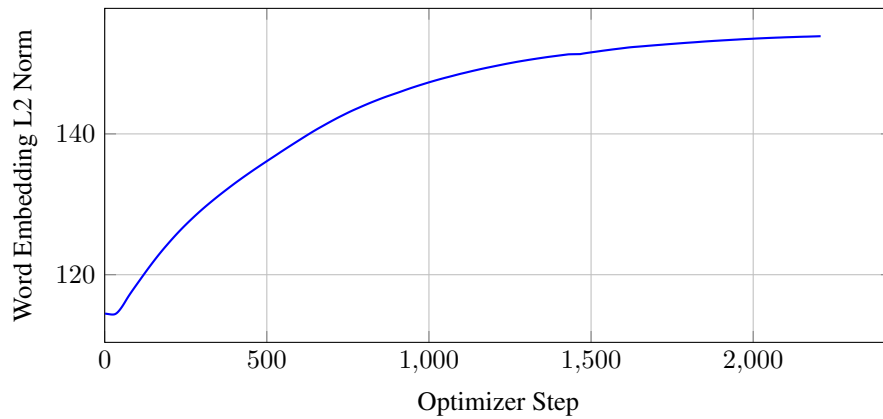


Figure 17: L2 Norm of embedding parameters during pretraining.

The training history, specifically train loss, validation loss, and gradient norm are shown in Figures 13 to 16. The validation loss was calculated on a smaller number of fineweb-edu evaluation samples. Due to the strong resource and time limitations, which limit the model training to 5 billion tokens, it is to be expected that the model will likely be undertrained. This is underlined by the word

embedding’s L2-norm only starting to stabilize / decrease towards the end of training, as seen in Figure 17.

We compare the training history of the same model being trained with a batch size of 1024 and with a batch size of 64 in Figure 18. Interestingly, while a higher batch size leads to better model performance than a smaller batch size for the same number of optimizer steps, it seems like the higher batch size negatively affects the validation loss with very low recurrent steps, specifically if only one recurrent step is done. This might be due to the richer gradient with batch size 1024 causing the model to learn a stronger refinement process of the latent tensor over multiple steps, which suffers stronger if only a single step is done, compared to a weaker refinement process that might be learned with a less dense gradient.

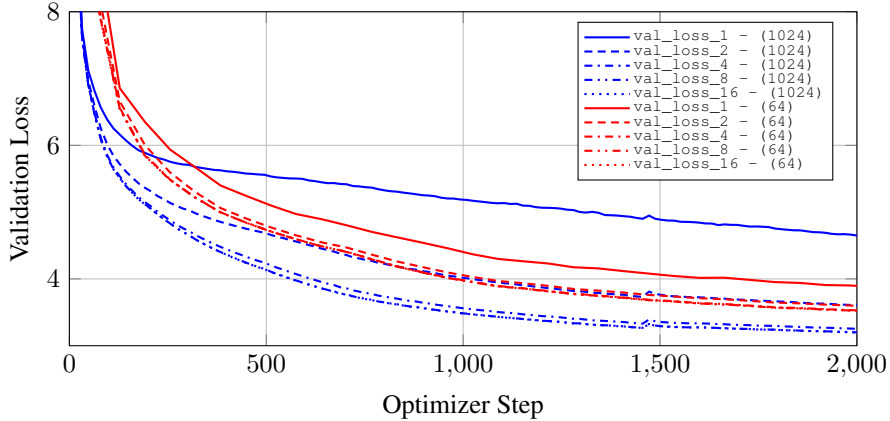


Figure 18: Comparison between a training run with batch size 1024 (blue) and batch size 64 (red) and different numbers of recurrent steps (1-16) for each.

4.3 BENCHMARKS

Table 11: Model performance comparison across benchmarks, compared to the original recurrent transformer block implementation by Geiping et al. (2025), as well as Amber (Liu et al., 2023), Pythia (Biderman et al., 2023), and OLMo (Groeneveld et al., 2024).

Model	Params	Tokens	ARC-C	HellaSwag	MMLU	WinoGrande
random	–	–	25.0	25.0	25.0	50.0
Amber	7B	1.2T	37.20	72.54	26.77	63.22
Pythia-2.8b	2.8B	0.3T	32.51	59.17	25.05	57.85
OLMo-1B	1B	3T	30.72	63.00	24.33	59.19
Raven, ($r = 4$)	3.5B	0.8T	27.99	43.46	23.39	55.24
Raven, ($r = 8$)	3.5B	0.8T	35.15	58.54	25.29	55.64
Raven, ($r = 16$)	3.5B	0.8T	37.71	64.67	31.25	57.77
Raven, ($r = 32$)	3.5B	0.8T	38.23	65.21	31.38	59.43
Novel, ($r = 1$)	0.3B	0.005T	0.1980	0.2680	0.2317	0.4964
Novel, ($r = 2$)	0.3B	0.005T	0.1766	0.2700	0.2480	0.5122
Novel, ($r = 4$)	0.3B	0.005T	0.1749	0.2718	0.2570	0.5209
Novel, ($r = 8$)	0.3B	0.005T	0.1826	0.2710	0.2545	0.5059
Novel, ($r = 16$)	0.3B	0.005T	0.1817	0.2717	0.2560	0.5004
Novel, ($r = 32$)	0.3B	0.005T	0.1817	0.2718	0.2552	0.5091
Novel, ($r = 64$)	0.3B	0.005T	0.1800	0.2728	0.2557	0.5114

After training the model on 5B tokens, it was evaluated on a set of benchmarks, shown in Table 11. The results underline the fact that the model is strongly undertrained, with scores very close to random guessing and, in the case of ARC-C, even worse than random guessing. For the reason that even with high compute utilization, e.g. 64 recurrent steps, the results are on the same level as random guessing, it does not make sense to evaluate how different numbers of recurrent steps for each of the three recurrent blocks affect the benchmark scores.

5 CONCLUSION

Due to resource limitations, the model is undertrained, making it impossible to judge the performance and quality of this architecture based on the benchmark scores. From the training history, however, we can make interesting and promising observations: It is demonstrated that stable model training is possible with multiple recurrent blocks. Further, it is shown that training with larger batch size promotes output quality for higher numbers of recurrent steps, at the cost of reduced output quality for very low numbers of recurrent steps.

This gives an overall positive outlook for future research possibilities regarding this architecture. These could include training a larger model on more data, and analyzing the benchmark scores of a not undertrained model. It would also be interesting to see if it is possible to pretrain the model with a lower number of recurrent steps and recalibrating it for more recurrent steps on a smaller dataset by either fully finetuning it or training a LoRA (Hu et al., 2022) on it, similarly to how it is done with YaRN (Peng et al., 2023b) for extending context length.

BIBLIOGRAPHY

- anon8231489123. ShareGPT_Vicuna_unfiltered. Hugging Face Datasets, 2023. URL https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered.
- Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics, 2023.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL <https://arxiv.org/abs/1607.06450>.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usven Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Yonatan Bisk, Rowan Zellers, Ronan Le bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):7432–7439, Apr. 2020. doi: 10.1609/aaai.v34i05.6239. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023. URL <http://jmlr.org/papers/v24/22-1144.html>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

codeparrot. github-code-clean. Hugging Face Datasets, 2022. URL <https://huggingface.co/datasets/codeparrot/github-code-clean>.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangan Zhou, Shanhua Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shut-ing Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xi-aokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers, 2019. URL <https://arxiv.org/abs/1807.03819>.

Wikimedia Foundation. Wikimedia downloads. URL <https://dumps.wikimedia.org>.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.

Bleys Goodson. Fine flan: Seqio to parquet so you don’t have to. <https://huggingface.co/datasets/Open-Orca/FLAN>, 2023.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind That-tai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Kore-vaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Ma-hadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu,

Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jong-soo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yearly, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Rapparthi, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie DelPierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leohardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippou Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liao, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent,

-
- Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Alex Graves. Adaptive computation time for recurrent neural networks, 2017. URL <https://arxiv.org/abs/1603.08983>.
- Dirk Groeneveld, Iz Beltagy, Evan Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, Tushar Khot, William Merrill, Jacob Morrison, Niklas Muennighoff, Aakanksha Naik, Crystal Nam, Matthew Peters, Valentina Pyatkin, Abhilasha Ravichander, Dustin Schwenk, Saurabh Shah, William Smith, Emma Strubell, Nishant Subramani, Mitchell Wortsman, Pradeep Dasigi, Nathan Lambert, Kyle Richardson, Luke Zettlemoyer, Jesse Dodge, Kyle Lo, Luca Soldaini, Noah Smith, and Hannaneh Hajishirzi. OLMo: Accelerating the science of language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15789–15809, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.841. URL <https://aclanthology.org/2024.acl-long.841/>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Tamir David Hay and Lior Wolf. Dynamic layer tying for parameter-efficient transformers, 2024. URL <https://arxiv.org/abs/2401.12819>.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021. URL <https://arxiv.org/abs/2009.03300>.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL <https://arxiv.org/abs/2203.15556>.
- Nathan Hoos, Wing Lian, Guan Wang, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknum". Slimorca dedup: A deduplicated subset of slimorca, 2023. URL <https://huggingface.co/datasets/Open-Orca/SlimOrca-Dedup/>.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

-
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis Bach and David Blei (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/ioffe15.html>.
- Nikhil Kandpal, Brian Lester, Colin Raffel, Sebastian Majstorovic, Stella Biderman, Baber Abbasi, Luca Soldaini, Enrico Shippole, A. Feder Cooper, Aviya Skowron, Shayne Longpre, Lintang Sutawika, Alon Albalak, Zhenlin Xu, Guilherme Penedo, Loubna Ben, Elie Bakouch, John David, Honglu Fan, Dashiell Stander, Guangyu Song, Aaron Gokaslan, John Kirchenbauer, Tom Goldstein, Brian R, Bhavya Kailkhura, and Tyler Murray. The Common Pile v0.1: An 8TB Dataset of Public Domain and Openly Licensed Text. *arXiv preprint*, 2025.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 12 2014.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2020. URL <https://arxiv.org/abs/1909.11942>.
- Zhengzhong Liu, Aurick Qiao, Willie Neiswanger, Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li, Yuqi Wang, Suqi Sun, Omkar Pangarkar, Richard Fan, Yi Gu, Victor Miller, Yonghao Zhuang, Guowei He, Haonan Li, Fajri Koto, Liping Tang, Nikhil Ranjan, Zhiqiang Shen, Xuguang Ren, Roberto Iriondo, Cun Mu, Zhiting Hu, Mark Schulze, Preslav Nakov, Tim Baldwin, and Eric P. Xing. Llm360: Towards fully transparent open-source llms, 2023. URL <https://arxiv.org/abs/2312.06550>.
- Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. Fineweb-edu: the finest collection of educational content, 2024. URL <https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu>.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering, 2018. URL <https://arxiv.org/abs/1809.02789>.
- Tomáš Mikolov, Stefan Kombrink, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5528–5531, 2011. doi: 10.1109/ICASSP.2011.5947611.
- Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math, 2024.
- nickrosh. Evol-instruct-code-80k-v1. Hugging Face Datasets, 2023. URL <https://huggingface.co/datasets/nickrosh/Evol-Instruct-Code-80k-v1>.
- OpenAI. Openai o1 system card. System card, OpenAI, December 2024. URL <https://cdn.openai.com/o1-system-card-20241205.pdf>. Accessed 2025-10-20.
- ostapeno. tinygsm-mind. Hugging Face Datasets, 2025. URL <https://huggingface.co/datasets/ostapeno/tinygsm-mind>.
- Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023.
- Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartłomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang,

-
- Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. Rwkv: Reinventing rnns for the transformer era, 2023a. URL <https://arxiv.org/abs/2305.13048>.
- Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models, 2023b. URL <https://arxiv.org/abs/2309.00071>.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- sahil2801. Codealpaca-20k. Hugging Face Datasets, 2023. URL <https://huggingface.co/datasets/sahil2801/CodeAlpaca-20k>.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782. doi: 10.1145/3474381. URL <https://doi.org/10.1145/3474381>.
- sedthh. gutenberg english. Hugging Face Datasets, 2023. URL https://huggingface.co/datasets/sedthh/gutenberg_english.
- Alex Singla, Alexander Sukharevsky, Lareina Yee, Michael Chui, and Bryce Hall. The state of ai in early 2024: Gen ai adoption spikes and starts to generate value. <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2024>, May 2024. Survey; QuantumBlack, AI by McKinsey. Accessed 2025-10-20.
- Luca Soldaini and Kyle Lo. peS2o (Pretraining Efficiently on S2ORC) Dataset. Technical report, Allen Institute for AI, 2023. ODC-By, <https://github.com/allenai/pes2o>.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijie Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Hao Hu, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Chao Hong, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Lijun Lu, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Xinjie Sun, Flood Sung, Heyi Tang, Jiawen Tao, Qifeng Teng, Chensi Wang, Dinglu Wang, Feng Wang, Haiming Wang, Jianzhou Wang, Jiaying Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Xiaofei Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yangkun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Huabin Zheng, Shaojie Zheng, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Zhen Zhu, Weiye Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2025. URL <https://arxiv.org/abs/2507.20534>.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.

-
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
- Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions, 2017. URL <https://arxiv.org/abs/1707.06209>.
- P.J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990. doi: 10.1109/5.58337.
- Ronald J. Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural Computation*, 2(4):490–501, 1990. doi: 10.1162/neco.1990.2.4.490.
- WizardLMTeam. WizardLM_evol_instruct_V2_196k. Hugging Face Datasets, 2023. URL https://huggingface.co/datasets/WizardLMTeam/WizardLM_evol_instruct_V2_196k.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In Hal Daumé III and Aarti Singh (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 10524–10533. PMLR, 13–18 Jul 2020. URL <https://proceedings.mlr.press/v119/xiong20b.html>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhen-guo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence?, 2019. URL <https://arxiv.org/abs/1905.07830>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/1e8a19426224ca89e83cef47f1e7f53b-Paper.pdf.

.1 MODEL OUTPUT EXAMPLES

This section shows example outputs from the model using different numbers of recurrent iterations per core block. The notation $[n_1, n_2, n_3]$ indicates the number of iterations for each of the three core blocks. Every text was generated with greedy sampling and an output token limit of 512. Repeating sections are truncated and replaced with [...] for readability.

Prompt:

Please write a python program that calculates the first 10 prime numbers.

Steps: [1, 1, 1]

To calculate the first 10 prime number number, we can calculate the first 10 prime numbers.

To calculate the first 10 prime numbers, we can calculate the first 10 prime numbers:

To calculate the first 10 prime numbers, we can calculate the first 10 prime numbers:

To calculate the first 10 prime numbers:

To calculate the first 10 prime numbers, we can calculate the first 10 prime numbers: [...]

Steps: [2, 2, 2]

Here is a Python program that calculates the first 10 prime numbers:

```
def is_prime(num):
    if num < 10:
        return False
    else:
        return False
```

```
def is_prime(num):
    if num < 10:
        return False
    else:
        return False
```

[...]

Steps: [4, 4, 4]

Here is a Python program that calculates the first 10 prime numbers:

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

[...]

Prompt:

Please write a python program that calculates the first 10 prime numbers. *(continued)*

Steps: [8, 8, 8]

Here is a Python program that calculates the first 10 prime numbers:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

[...]
```

Steps: [16, 16, 16]

Here is a Python program that calculates the first 10 prime numbers:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

[...]
```

Steps: [32, 32, 32]

<same as above>

Prompt:

Please write a java program that calculates the first 10 prime numbers.

Steps: [1, 1, 1]

To find the number of numbers, we can calculate the first 10 prime numbers:

Prompt:

Please write a java program that calculates the first 10 prime numbers. *(continued)*

To find the number of numbers, we can calculate the number of numbers:

To find the number of numbers, [...]

Steps: [2, 2, 2]

Here is a java program that calculates the first 10 prime numbers:

```
import java.util.Scanner;

public class PrimeNumbers {
    public static void main(String[] args) {
        int[] numbers = new int[] args;
        int[] numbers = new int[] args;
        int[] numbers = new int[] args;
        int[] numbers = new int[] args;
        int[] numbers = new int[] args;

        for (int i = 0; i < numbers.length; i++) {
            if (i == 0) {
                return 0;
            }

        }

        for (int i = 0; i < numbers.length; i++) {
            if (i == 0) {
                return 0;
            }

        }
        [...]
    }
}
```

Steps: [4, 4, 4]

Here is a Java program that calculates the first 10 prime numbers:

```
import java.util.ArrayList;
import java.util.List;

public class PrimeNumbers {
    public static void main(String[] args) {
        int[] primeNumbers = new int[] { 10, 10, 10, 10, 10, 10, 10
        [...]
    }
}
```

Steps: [8, 8, 8]

Here is a Java program that calculates the first 10 prime numbers:

```
import java.util.Scanner;

public class PrimeNumbers {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Check if the number is prime
    }
}
```

Prompt:

Please write a java program that calculates the first 10 prime numbers. *(continued)*

```
        if (isPrime(scanner)) {
            System.out.println("The_number_is_prime");
        } else {
            System.out.println("The_number_is_not_prime");
        }
    }
}
```

In this program, we use a 'Scanner' class to read the number from the user. We use a 'Scanner' class to read the number from the user. We then check if the number is prime using the 'isPrime' method. If it is, we print the number.

Steps: [16, 16, 16]

Here is a Java program that calculates the first 10 prime numbers:

```
import java.util.Scanner;

public class PrimeNumbers {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Check if the number is prime
        if (isPrime(scanner)) {
            System.out.println("The_number_is_prime");
        } else {
            System.out.println("The_number_is_not_prime");
        }

        // Check if the number is prime
        if (isPrime(scanner)) {
            System.out.println("The_number_is_not_prime");
        } else {
            System.out.println("The_number_is_not_prime");
        }
        [...]
    }
}
```

Steps: [32, 32, 32]

<same as above>

Prompt:

Please write a cpp program that calculates the first 10 prime numbers.

Steps: [1, 1, 1]

The first 10 prime numbers is 10 prime numbers.

The first 10 prime prime numbers are 10 prime numbers.

The first 10 prime prime prime numbers are: 10 prime prime prime prime prime numbers.

The first 10 prime prime prime prime numbers are: 10 prime prime prime prime [...]

Steps: [2, 2, 2]

To calculate the first 10 prime numbers, we need to find the first prime number and then divide it by the other number.

Prompt:

Please write a cpp program that calculates the first 10 prime numbers. *(continued)*

First, we divide the first prime number by the other number and then divide it by the other number.
Next, we divide the first prime number by the other number and then divide it by the other number.
Next, we divide the first prime number by the other number and then divide it by the other number and then divide it by the other number. [...]

Steps: [4, 4, 4]

Here is a cpp program that calculates the first 10 prime numbers:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdlib.h>
#include <stdlib.h>
[...]
```

Steps: [8, 8, 8]

Here is a cpp program that calculates the first 10 prime numbers:

```
#include <stdio.h>

int main() {
    int num10;
    int num20;

    // Calculate the first 10 prime numbers
    int first10 = 10;
    int second10 = 10;

    // Calculate the first 10 prime numbers
    int first10 = 10;
    int second10 = 10;

    [...]
```

Steps: [16, 16, 16]

Here is a cpp program that calculates the first 10 prime numbers:

```
#include <stdio.h>

int main() {
    int num10;
    int num20;

    // Calculate the first 10 prime numbers
    int first10 = 10;
    int second10 = 10;

    // Calculate the first 10 prime numbers
    int first10 = 10;
    int second10 = 10;

    [...]
```

Prompt:

Please write a cpp program that calculates the first 10 prime numbers. *(continued)*

Steps: [32, 32, 32]

<same as above>

Prompt:

Please tell me funny cat facts.

Steps: [1, 1, 1]

A person who is a person who is a person who is a person who is a person who [...]

Steps: [2, 2, 2]

The cat's cat's cat's cat's [...]

Steps: [4, 4, 4]

The cat is a type of cat, and it is a type of cat.

Steps: [8, 8, 8]

Sure! Here's a funny cat story about a cat named "Sure! It's a funny cat!"

Steps: [16, 16, 16]

Sure! Here's a funny cat story about a cat named "The Cat" and a funny cat named "The Cat" in the story: "The Cat" is a funny cat named "The Cat" and a funny cat named "The Cat" is a funny cat named "The Cat" in the story.

Steps: [32, 32, 32]

<same as above>

Prompt:

How to bake a tasty fruit cake:

Steps: [1, 1, 1]

To bake a tasty fruit fruit fruit [...]

Steps: [2, 2, 2]

To bake a tasty fruit cake, you can use a combination of 20000 tasty fruit cake.

To bake a tasty fruit cake, you can use a combination of 20000 tasty fruit cake.

To bake a tasty fruit cake, you can use a combination of 20000 tasty fruit cake. [...]

Steps: [4, 4, 4]

1. Eat a tasty fruit cake. 2. Eat a tasty fruit cake. 3. Eat a tasty fruit cake.

1. Eat a tasty fruit cake. 2. Eat a tasty fruit cake.

1. Eat a tasty fruit cake.

1. Eat a tasty fruit cake.

Steps: [8, 8, 8]

1. Preheat the tasty fruit cake to 200°F. 2. Pour the tasty fruit cake into a small bowl. 3. Pour the tasty fruit cake into a small bowl. 4. Pour the tasty fruit cake into a small bowl. 5. Pour the tasty fruit cake into a small bowl. 6. Pour the tasty fruit cake into a small bowl. 7. Pour the tasty fruit cake into a small bowl. 8. Pour the tasty fruit cake into a small bowl.

1. Preheat the tasty fruit cake to 200°F. 2. Pour the tasty fruit cake into a small bowl. 3. Pour the tasty fruit cake into a small bowl. 4. Pour the tasty fruit cake into a small bowl.

Prompt:
How to bake a tasty fruit cake: (continued)

Steps: [16, 16, 16]

1. Bake a tasty fruit cake: 2. Bake a tasty fruit cake: 3. Bake a tasty fruit cake: 4. Bake a tasty fruit cake:
5. Bake a tasty fruit cake: 6. Bake a tasty fruit cake: 7. Bake a tasty fruit cake: 8. Bake a tasty fruit cake:
Bake a tasty fruit cake:
Bake a tasty fruit cake: [...]

Steps: [32, 32, 32]

<same as above>

Prompt:
What are recent world news?

Steps: [1, 1, 1]

The question is:
The question is:
The answer is:
The question is: [...]

Steps: [2, 2, 2]

The United States has a total of 17,000 people, and the United States has a total of 17,000 people. The United States has a total of 17,000 people, and the United States has a total of 17,000 people. The United States has a total of 17,000 people, and the United States has a total of 17,000 people. The United States has a total of 17,000 people, and the United States has a total of 17,000 people, and the United States has a total of 17,000 people, and the United States has a total of 17,000 people.

Steps: [4, 4, 4]

The United States has a long-term trend in the global news. The United States has a long-term trend in the global news. The United States has a long-term trend in the global news.

Steps: [8, 8, 8]

The United Nations (UN) has a long-term plan to address the global crisis in the world. The United Nations (UN) has a long-term plan to address the global crisis in the world. The United Nations (UN) has a long-term plan to address the global crisis in the world. [...]

Steps: [16, 16, 16]

The United Nations (UN) has a long-term plan to address the global crisis in the world. The United Nations (UN) has a long-term plan to address the global crisis in the world. [...]

Steps: [32, 32, 32]

<same as above>

Prompt: (Temperature: 0.7)
How to make a tasty pasta:

Steps: [12, 12, 12]

1. Preheat your your mouth and avoid using a hot potato soda and a hot sauce.
2. Clean the vegetable garden by removing any unplugging vegetables.
3. Pour the vegetables in a warm-blooded and nutritious dish, such as garlic, nuts, seeds, and nuts.
4. Use a cool or cold bath, such as a bath, bath, or a bath.

Prompt: (Temperature: 0.7)
How to make a tasty pasta: (*continued*)

5. Pour the vegetables in a cool, dry and spicy dish, such as a toast or salad.

To make a tasty pasta, simply pour the vegetables in a warm-blooded and nutritious dish, such as a toast or salad, with the vegetables in the dish cool, and then, pour the vegetables in a cool, dry, and spicy dish.

Steps: [12, 12, 12]

1. Tasty pasta: A tasty pasta is a dish made with a mixture of fruits, vegetables, and veggies. It has a mixture of fruits, vegetables, and veggies.

2. Tasty pasta: A dish made with a mixture of fruits, vegetables, and veggies has a mixture of fruits, vegetables, and veggies. It has a mixture of fruits, vegetables, and veggies and is a good choice for those looking to enjoy a healthy pasta.

Steps: [12, 12, 12]

To make a tasty pasta, we can do both:

1. Keep the vegetables in a large bowl and add a dishwasher to the dishwasher to wash away the vegetables.
2. Brush the vegetables in a cup of water.
3. Toss the vegetables in a dishwasher.
4. Pour the vegetables in a bowl.

Toss the vegetables in the dishwasher.

1. Remove the vegetables from the dishwasher and remove any vegetables from the dishwasher.

1. Pour the vegetables in a bowl and fill it with the vegetables.

2. Cut the vegetables in the dishwasher and put the vegetables in the bowl and put it in the dishwasher and put it in the bowl and put it in the bowl and put it in the dishwasher.

3. Push the vegetables in a bowl and place it in a bowl.

4. Brush the vegetables in the bowl and put it in the bowl and put it in the bowl and put it in the bowl.

5. Pour the vegetables in a bowl and put it in the bowl and put it in the bowl and put it in the bowl. [...]